# Software Construction & Development

Topic 1

# What is  Construction?

# What is Construction?

**Construction** Noun

"the process, art, or manner of building something"

Merriam-Webster

"the work of building or making something"

Cambridge Dictionary

# Software Engineering Body of Knowledge (SWEBOK) major Knowledge Areas?

**Software requirements**

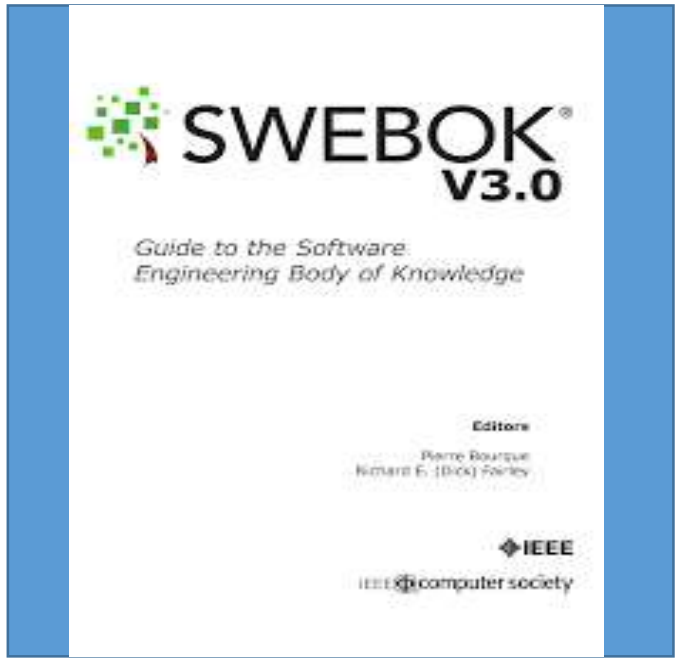
**Software design**

**Software construction**

**Software testing**

**Software maintenance**

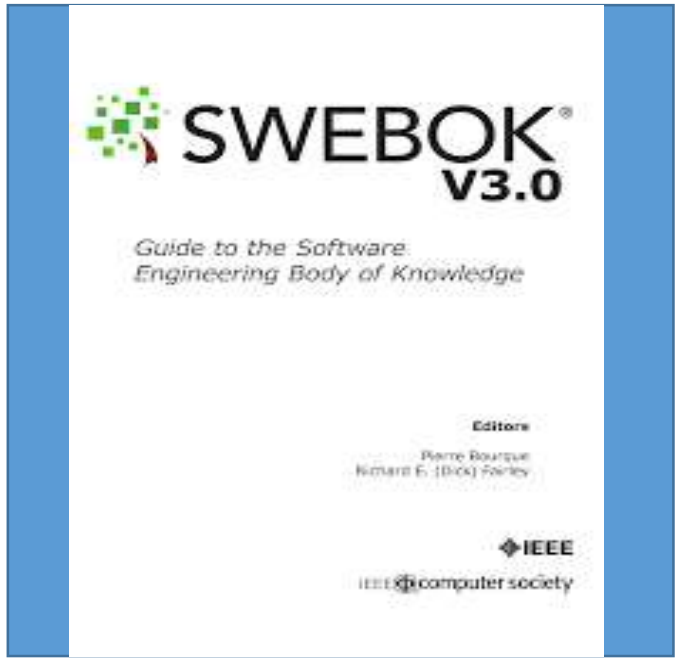
ISO/IEC TR 19759



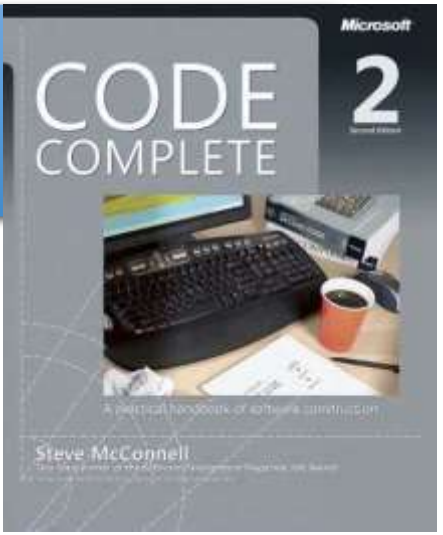
# What is Software Construction?

# What is Software Construction?

**The term software construction refers to the detailed creation of working software through a combination of coding, verification, unit testing, integration testing, and debugging. (SWEBOK)**



# What is Software Construction?




**Creating software through combination of detailed design, construction planning, coding, debugging, unit testing, integration testing, and considering corrective maintenance.**



*Steve McConnell*
*Winner of Software Development Productivity Award*

*Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.*

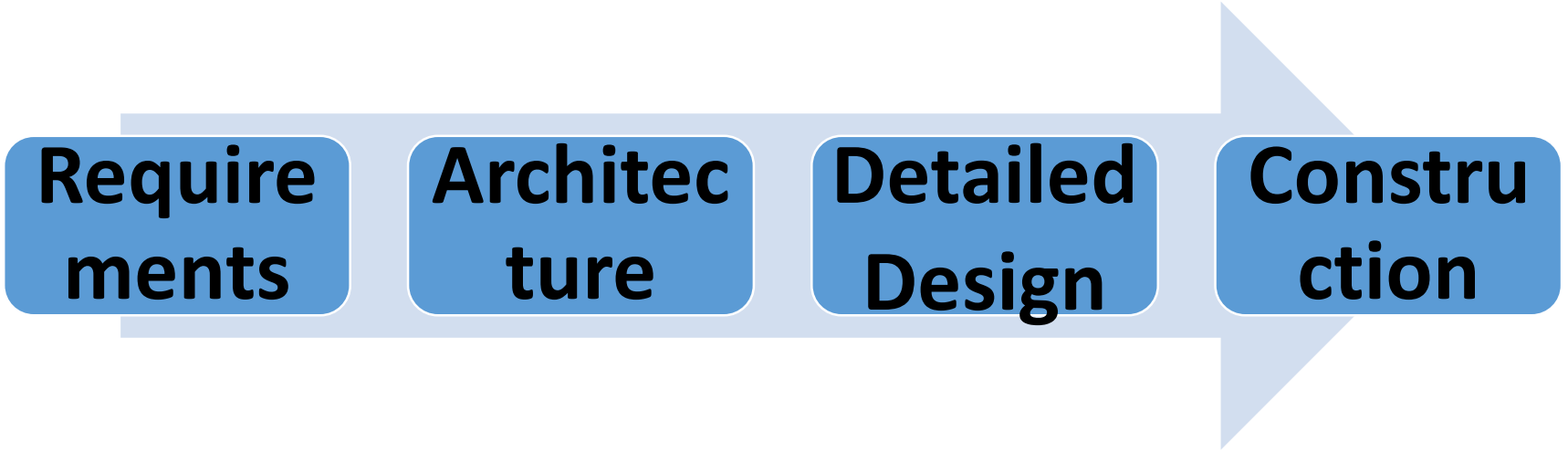
# Software Construction: objective

- **Objective is to map detailed design on to correct and manageable code**

*Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.*

# Software Construction: when to start

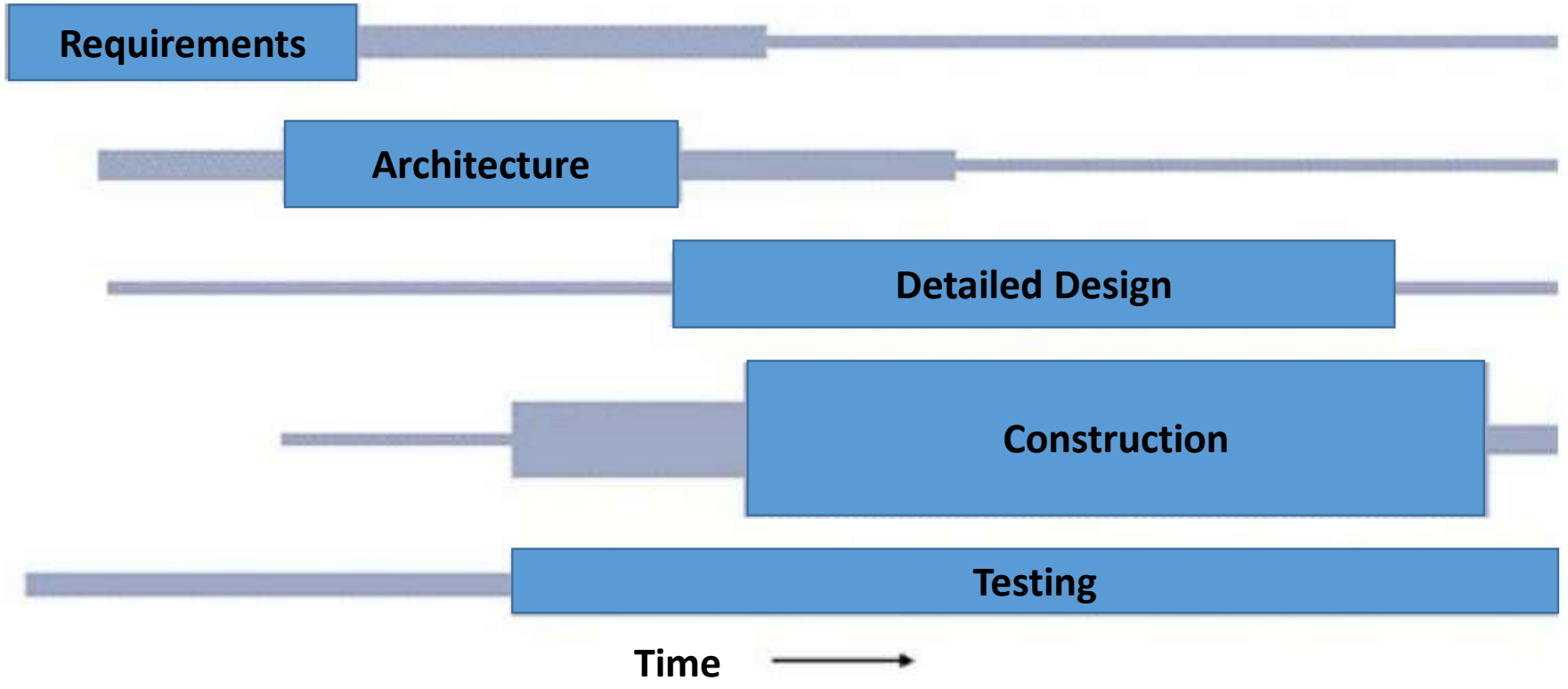
- **Software Construction is an activity of software development started after requirement engineering, architecture and detailed design**




*Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.*

# Overlapping Software Construction with Other Activities

**Activities overlapped to some degree on most projects**




*Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.*

# Software Construction Activities

# Software Construction Activities
# Topic 2

# Software Construction Activities

**The major activities in construction are**

**Coding**

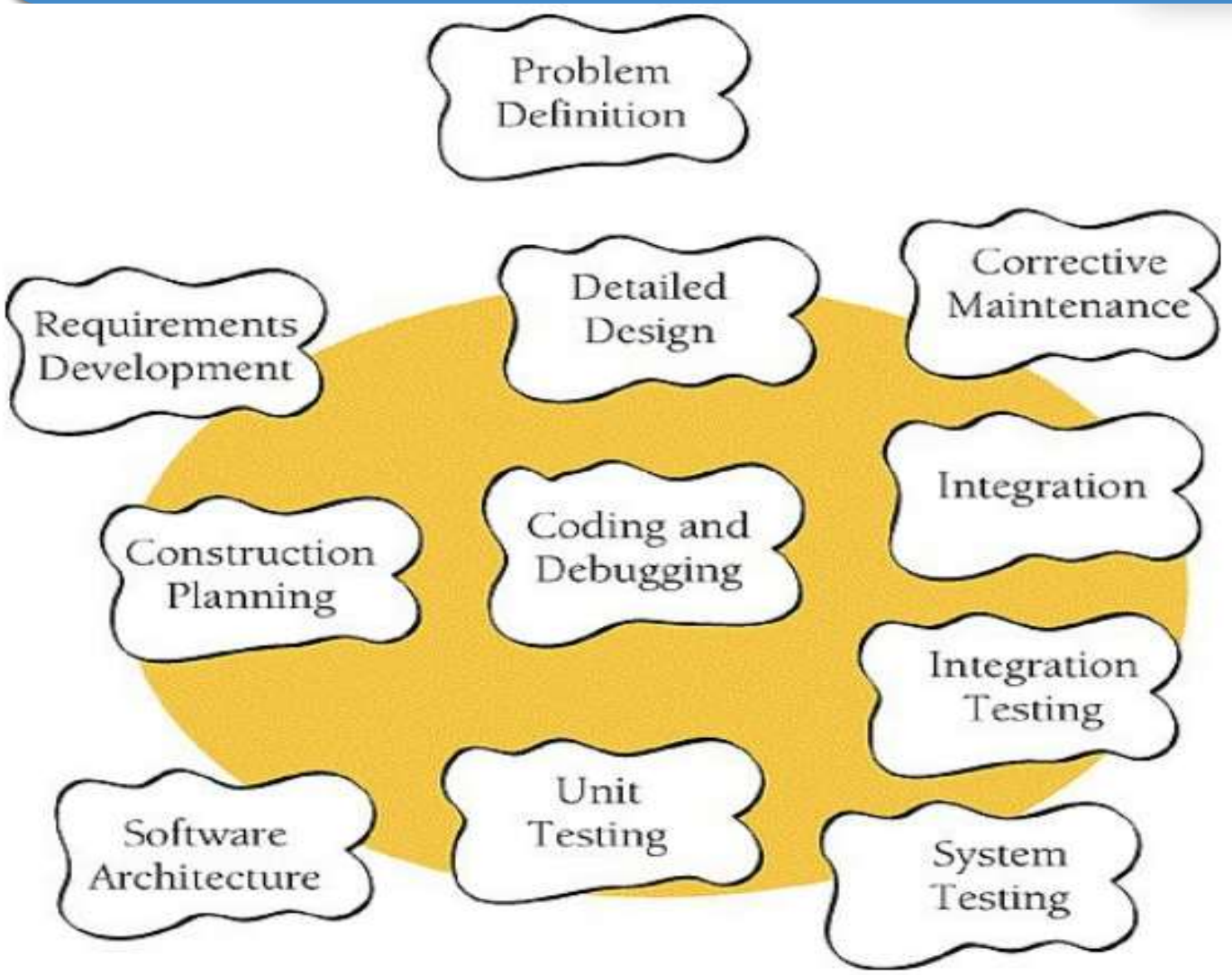
**debugging**

**integration**

**developer testing (unit testing and integration testing)**

**and continuous liaison with detailed design.**

# Software Construction Activities




**Construction Activities (inside circle) focuses on coding and debugging but also includes detailed design, unit testing, integration testing, construction planning and other activities**

***Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.***

# Specific Tasks involved in Software Construction Activities

1. **Designing and writing classes, methods and relationships**
2. **Creating and naming variables and named constants**
3. **Selecting control structures and organizing blocks of statements**
4. **Polishing code by carefully formatting and commenting**
5. **Code safety, Reliability and ethics**
6. **Ensure compliance to good Software Coding Standard**

   **e.g. MISRA C++**

# Specific Tasks involved in Software Construction Activities

7. **Verifying that the code build according to specification**
8. **Determining how your code will be tested**
9. **Integrating software components that were created separately**
10. **Tuning code to make it faster and use fewer resources**

# Problem Definition, Requirement Development and Construction Planning

**TOPIC 3**

# Major Software Development Activities

- **Problem definition**
- **Requirements development**
- **Construction planning**
- Software architecture, or high-level design
- Detailed design
- Coding and debugging
- Unit testing
- Integration testing
- Integration
- System testing
- Corrective maintenance

# Problem Definition

**Considered to be prerequisite of Software Construction**

**Clear statement of the problem that the system is supposed to solve**

Sometimes called “**Product Vision**” Or “**mission Statement**”

# Good and Poor Problem Statement

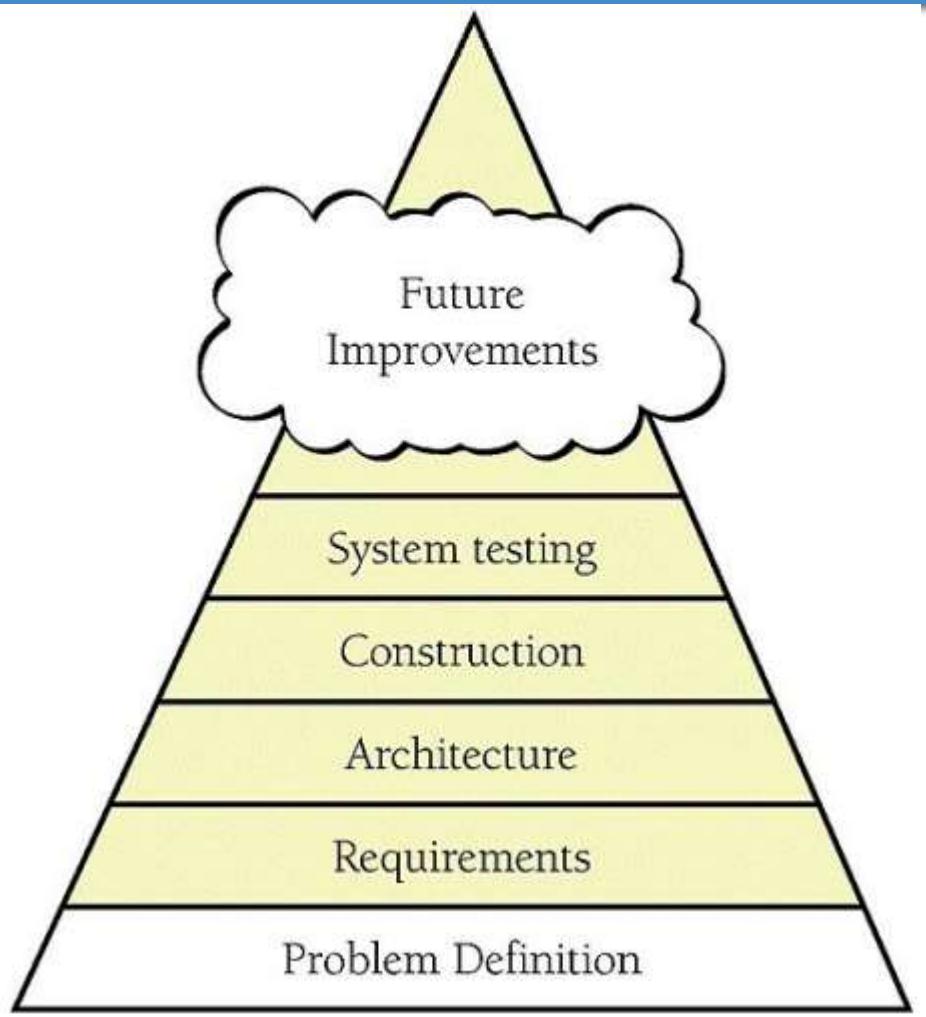
**A problem definition defines what the problem is without any reference to possible solutions.**

**Example 1: We can not keep up with orders for the IBM** (Good)

**Example 2: We need to introduce Blockchain based distributed digital ledger system to keep up with orders for the IBM (Poor)**

**It doesn't sound like a problem; it sounds like a solution**

# Problem definition as Foundation of Development Process




***Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.***

# Requirement Development

Requirements are statements of what the system must do, how it must behave, the properties it must exhibit, the qualities it must possess, and the constraints that the system and its development must satisfy.

SEI.CMU

# Managing Construction

- **Construction in Life Cycle Models**

  Linear vs Iterative models

- **Construction Planning**

  Order in which software components are created, allocation of tasks, quality

- **Construction Measurement**

  code developed, code modified, code reused, code destroyed, code complexity, code inspection statistics, fault-fix and fault-find rates, effort, and scheduling.

# Software Architecture, High-Level and Detailed Design

**TOPIC 4**

# Major Software Development Activities

- Problem definition
- Requirements development
- Construction planning
- **Software architecture, or high-level design**
- **Detailed design**
- Coding and debugging
- Unit testing
- Integration testing
- Integration
- System testing
- Corrective maintenance

# Software Architecture

"the set of structures needed to reason about the system, which comprise software elements, relations and properties" SWEBOK

**Big Picture : What, Where, but not how**

# Software Architecture Vs High Level Design

**Architecture refers to design constraints that apply system wide**

**whereas**

**High-level design refers to design constraints that apply at the subsystem or multiple-class level, but not necessarily system wide (SWEBOK)**

**Architecture is also known as "system architecture," "high-level design," and "top-level design."**

# Detailed Design

**Refines the elements of the software architecture and provides a detailed blueprint to the programmers responsible for the implementation of the software**

e.g. class diagrams, ER model, sequence diagrams, activity diagrams, naming conventions, data storage specification, I/O format

**Reverse and Forward Engineering Principle**

# Software Structure and Architecture

**1-Architectural Structures and Viewpoints**

**2- Architectural Styles**

**3- Design Patterns**

**4- Architecture Design Decisions**

**5-Families of Programs and Frameworks**

# Software Structure and Architecture

## 1-Architectural Structures and Viewpoints

**(logical view, Process view, Physical view, deployment view)**

# Software Structure and Architecture

## 2- Architectural Styles

- General Structure (Layers, pipes, filter)
- Distributed Systems (client/server, three-tiers, broker)
- Interactive Systems (microkernel, reflection)
- Adaptable (batch, interpreters, process control, rule based)
- Peer to Peer e.g. blockchain

# Software Structure and Architecture

## 3- Design Patterns

- Creational Patterns (Factory, Prototype, Builder, Singleton)
- Structural Patterns (Adaptor, Façade, Proxy)
- Behavioral Patterns (observer, state, iterator)

# Software Structure and Architecture

## 4- Architecture Design Decisions

- Trade-off among quality attributes  e.g Time-Space trade Off, sentinel Approach

## 5-Families of Programs and Frameworks

- designing reusable and customizable components

# Software Structure and Architecture

## 5-Families of Programs and Frameworks

- designing reusable and customizable components

# Coding and Debugging

**Topic 6**

# Major Software Development Activities

- Problem definition
- Requirements development
- Construction planning
- Software architecture, or high-level design
- Detailed design
- **Coding and debugging**
- Unit testing
- Integration testing
- Integration
- System testing
- Corrective maintenance

# Coding

**Coding is an activity of writing instruction in some programming language**

**“software construction is mostly coding and debugging”** SWEBOK

# Expertise required for Coding

- knowledge of the application domain
- appropriate data structures
- specialized algorithms
- language constructs
- good programming practices
- Knowledge of Coding Standards

# What is Bug?

a bug is an error in the source code that causes a program to produce unexpected results or crash



# What is Debugging?

**“Debugging is the activity to find and fix bugs (faults) in the source code (or design)”** **(SWEBOK)**

**Debugging Techniques**

- **Static Debugging (Code Review)**
- **Dynamic Debugging (closely associated with testing)**
- **Postmortem Debugging (Core dump of a process)**

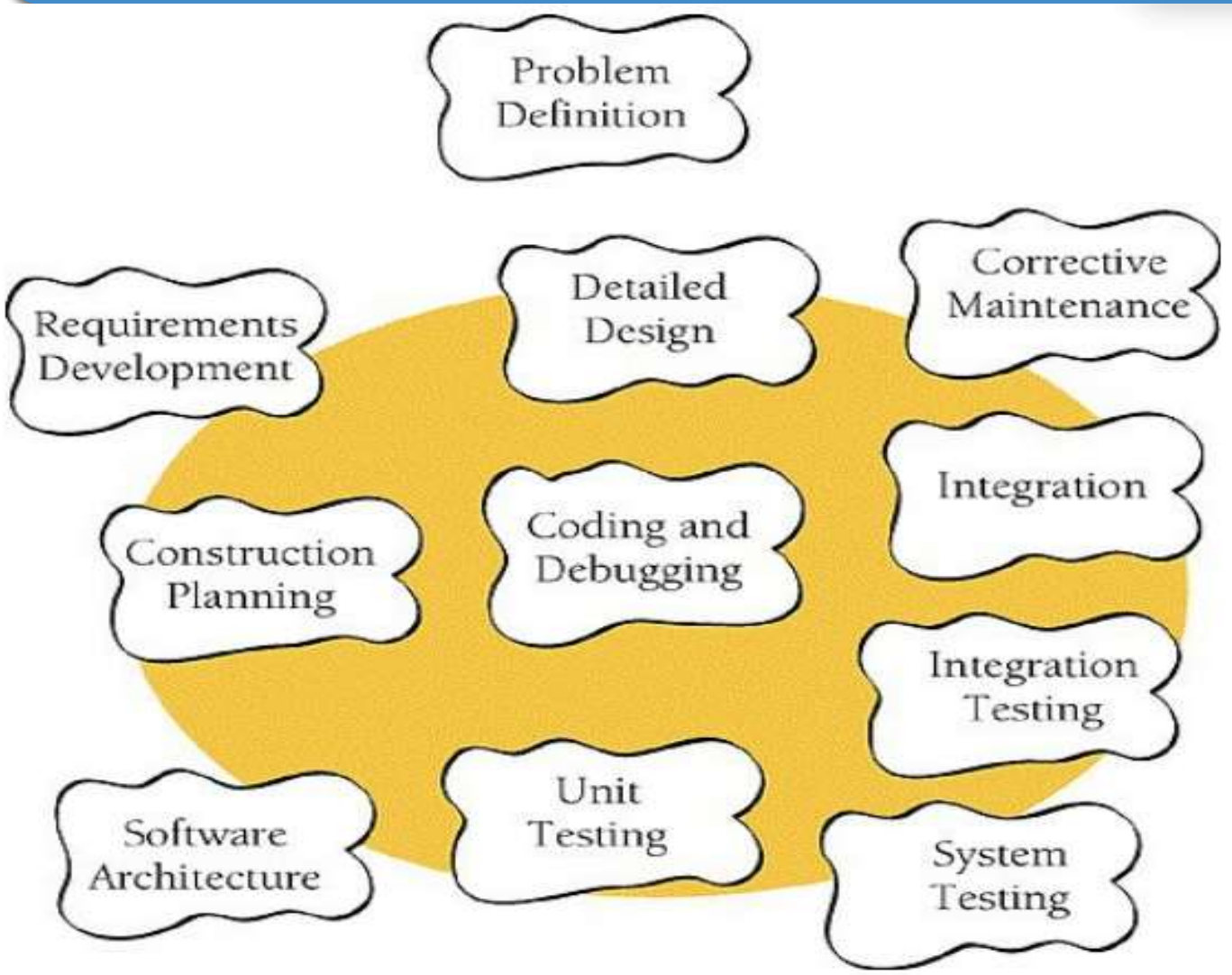
# Unit and Integration Testing
# Topic 06

# Major Software Development Activities

- Problem definition
- Requirements development
- Construction planning
- Software architecture, or high-level design
- Detailed design
- Coding and debugging
- **Unit testing**
- **Integration testing**
- Integration
- System testing
- Corrective maintenance

# What is Unit Testing ?

# Software Construction Activities




**Construction Activities (inside circle) focuses on coding and debugging but also includes detailed design, unit testing, integration testing, construction planning and other activities**

***Code Complete: A Practical Handbook of Software Construction, By Steve McConnell, Second Edition, Microsoft Press, 2019.***

# Unit Testing (Activity of Software Construction)

- Testing of smallest piece of code that can be logically isolated

  e.g. class, function, component

- Carried out by developers

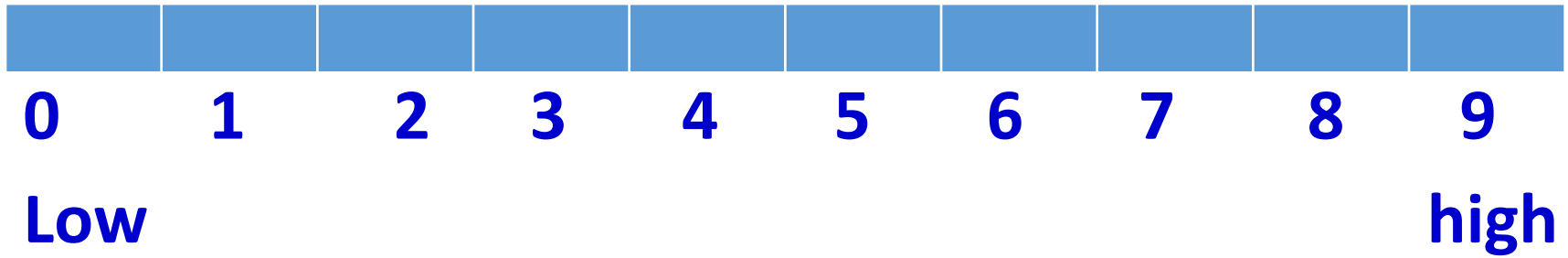
**Goal:** standalone module is functionally correct and fit for use

# Example : Binary Search (Unit test)

```
bool BinarySearch (A, key, low, high)
mid ← (low + high)/2
While ( key!=a[mid]) and ( low<high)
        if (key >a[mid])
             low= mid +1;
        else
              high = mid-1
        mid ← (low + high)/2
If ( low <high )
  return true
else
return false
```

# Example : Binary Search (Unit test)

Array




```
bool BinarySearch (A, key, low, high)
```

# Example : Binary Search (Possible Inputs and Expected Output)

| Possible Test Case | Input | Expected Output |
|---|---|---|
| 1.Array of size 1, element in array | [10] key=10 | true |
| 2.Array of size 1, element not in Array | [20] key=10 | false |
| 3.Even Array size , 1st element in Array | [10,20,30,40] key=10 | true |
| 4.Even Array size , Last element in Array | [10,20,30,40] key=40 | true |
| 5.Even Array size , not first or last element in Array | [10,20,30,40] key=50 | false |
| 6.Even Array size , element not in Array | | |
| 7.Odd Array size , 1st element in Array | | |
| 8.Odd Array size , Last element in Array | | |
| 9.Odd Array size , not first or last element in Array | | |
| 10.Odd Array size , element not in Array | | |

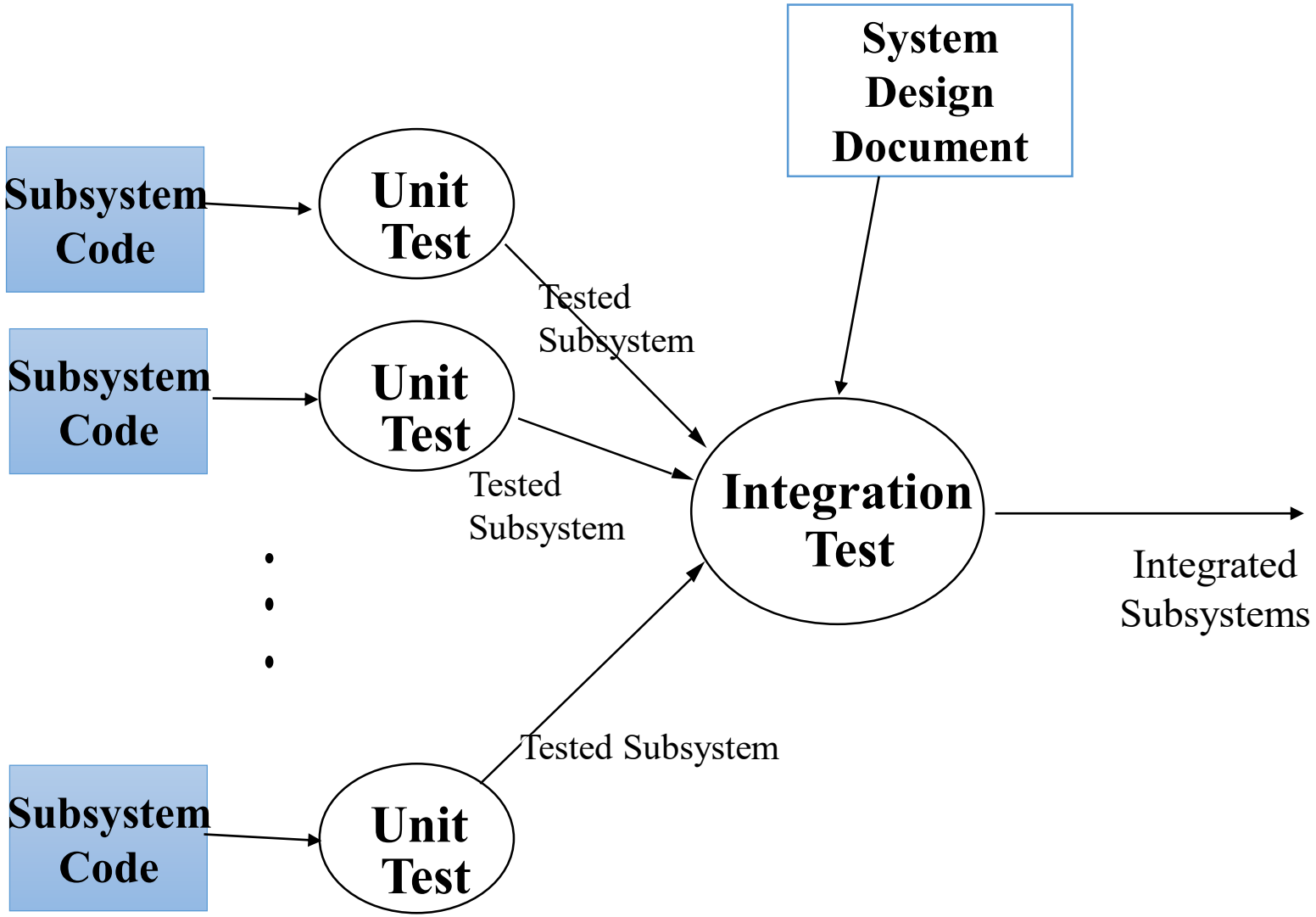
# Example : Binary Search (Unit test case in C++)

```
bool UnitTestCase ( ){
      bool flag =true;  int *a; int key=10;
      a= new int [1]; a[0]=key;
      if ( BinarySearch(a,key,0,1)!= true) // Possible Test Case 1
            flag=flase;
      a[0]=20;
      if ( BinarySearch(a,key,0,1)!=false) // Possible Test Case 2
           flag=flase;
     delete []a; a= new int[4]; a[0]=key, a[1]=20,   a[2]=30;a[3]=40;
     if (BinarySearch(a,key,0,4)!=true) // Possible Test Case 3
          flag=flase;
        ..................
      return flag
}
```

# Integration Testing (Activity of Software Construction)

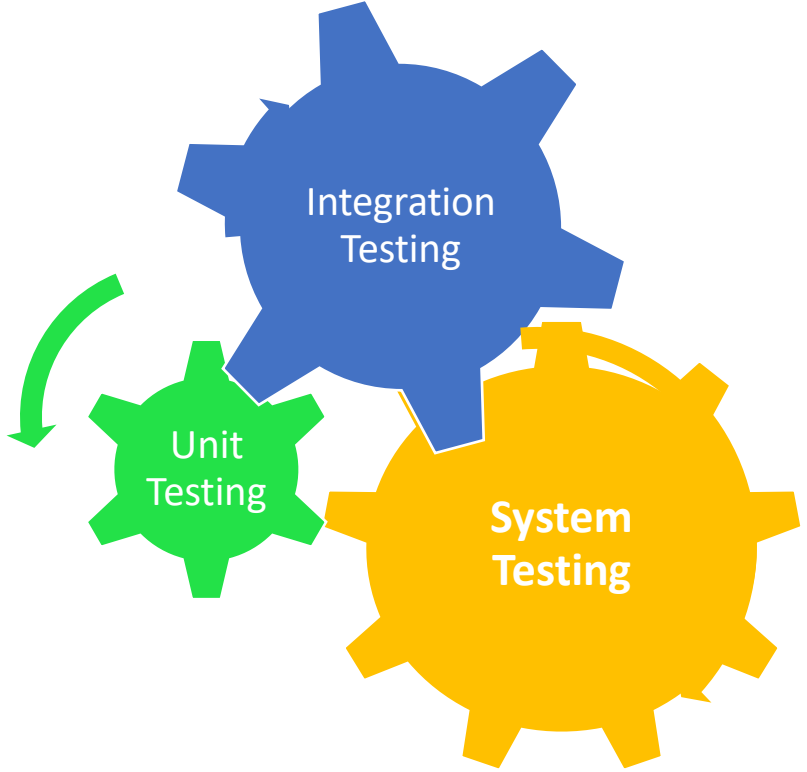
- Testing groups of subsystems (collection of classes, packages, components create by multiple programmer or teams)
- Carried out by developers

Goal: Test the interface among the subsystem




**All tests by developer**

# System Testing Topic



# Major Software Development Activities

- Problem definition
- Requirements development
- Construction planning
- Software architecture, or high-level design
- Detailed design
- Coding and debugging
- Unit testing
- Integration testing
- **System testing**
- Corrective maintenance

# System Testing

- test entire system behavior as a whole, with respect to scenarios and requirements

- execution of the software in its final configuration

- tests for security, performance, scalability, reliability, security and other issues

# Importance of System Testing




**Plane crash due to software failure-**Asiana Flight 214

Source: NBCNews.com

# Importance of System Testing



**Apple Engineer died in Tesla Crash Had Previously Complained About Autopilot**

**Source:** *Credit: Eric Marrapodi/CNN)*

# Importance of System Testing




**Software bug led to death of pedestrian in Uber's self-driving crash**

**Source: arstechnica.com**

# Importance of System Testing



**Malfunction due to software, leading to 3 dead and 3 critically injured**

**Therac-25 radiation machine used to treat cancer patients to remove tumor**

**Source : http://kalfaoglu.com**

# Importance of System Testing

**Failed Satellite launch – 1.2 $ billion Lost**

**Source: NASA**



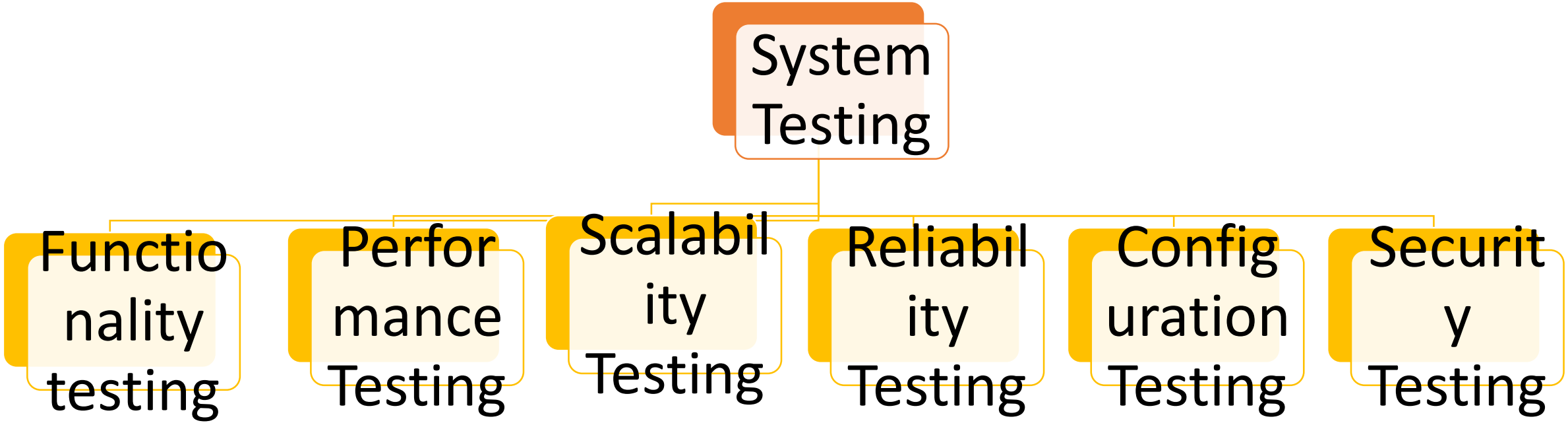
# Importance of System Testing



**Public launch of Windows 98 Fails**

**Source: Vuusteri**

# System Testing Types



# Software Construction & Development

Week-2

# Software development Process

- In software engineering, a software development process is a process of dividing software development work into smaller, parallel, or sequential steps or sub-processes to improve design and/or product management.
- Provides a roadmap to software development
- Types of Software development Processes
  - Conventional
  - Agile

# Steps in the software development process




https://www.spaceotechnologies.com/blog/software-development-process/

# Software Development Process Phase 1

## 1. Requirement Gathering & Analysis Phase

**Goal:** Document functional and non-functional requirements(NFRs).

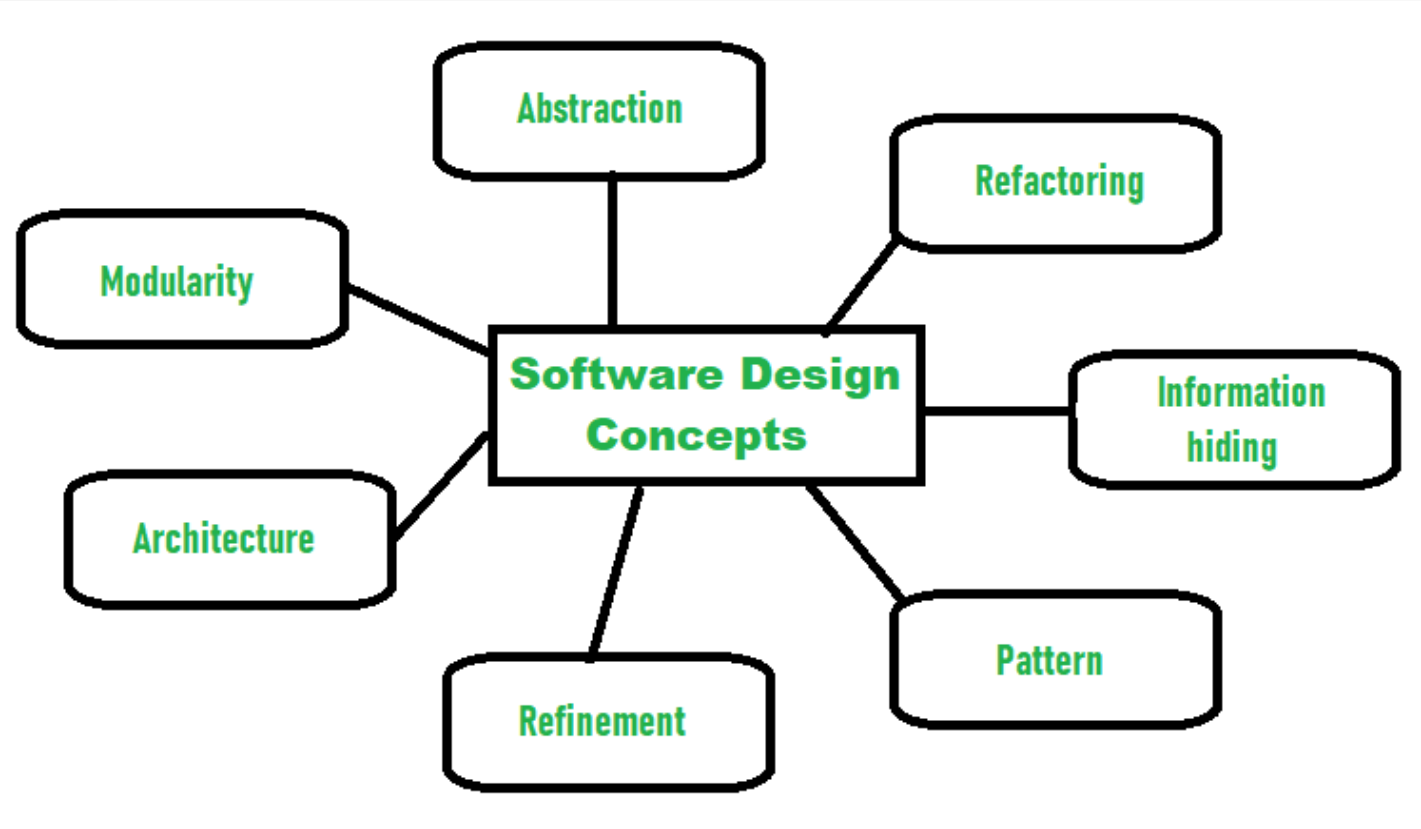
**Who:** Business Analyst, Functional/Business Users, Architect(s), Lead Developer, Project Manager or Scrum Master.

**Process:** Stakeholder discussions, brainstorming sessions, interviews, Prototyping, Role playing, Review existing system/process.

**Deliverables:** identified Actors, UseCases, Screen Prototypes, UX designs. Reviewed and Approved requirements.

-by SFSupport247.com

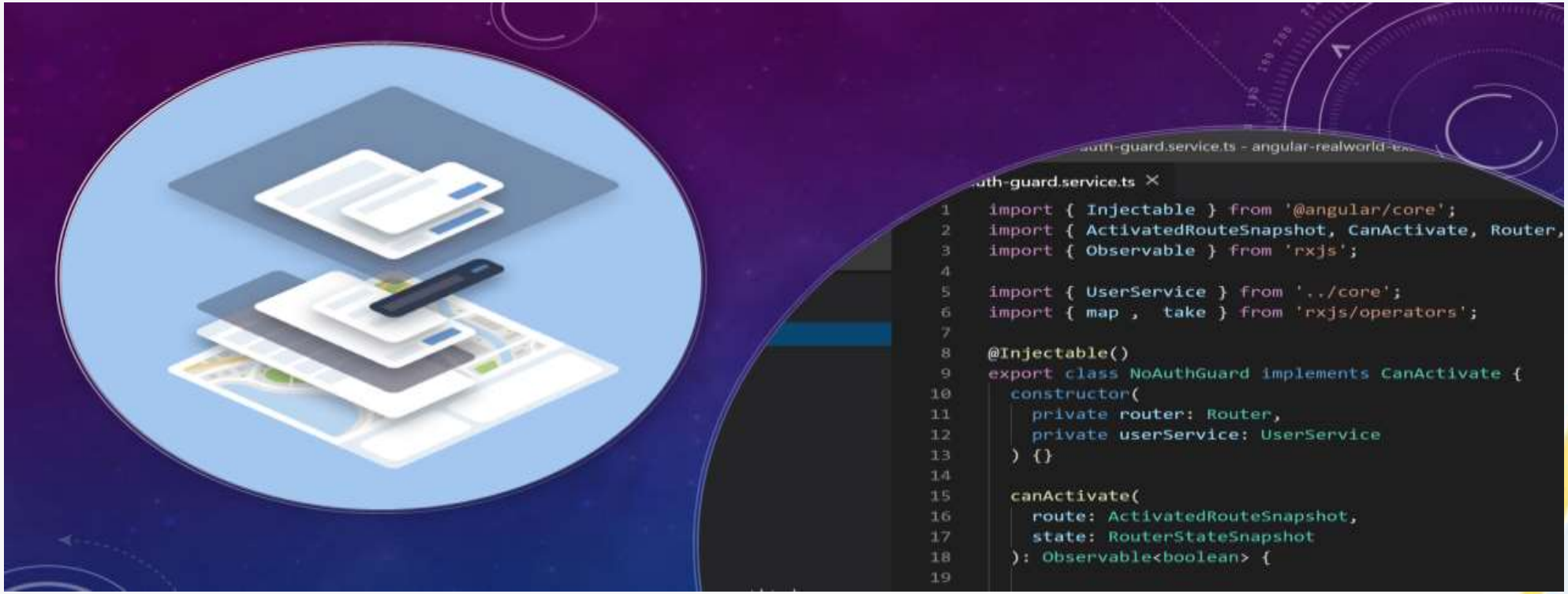
# Software Design and Prototyping



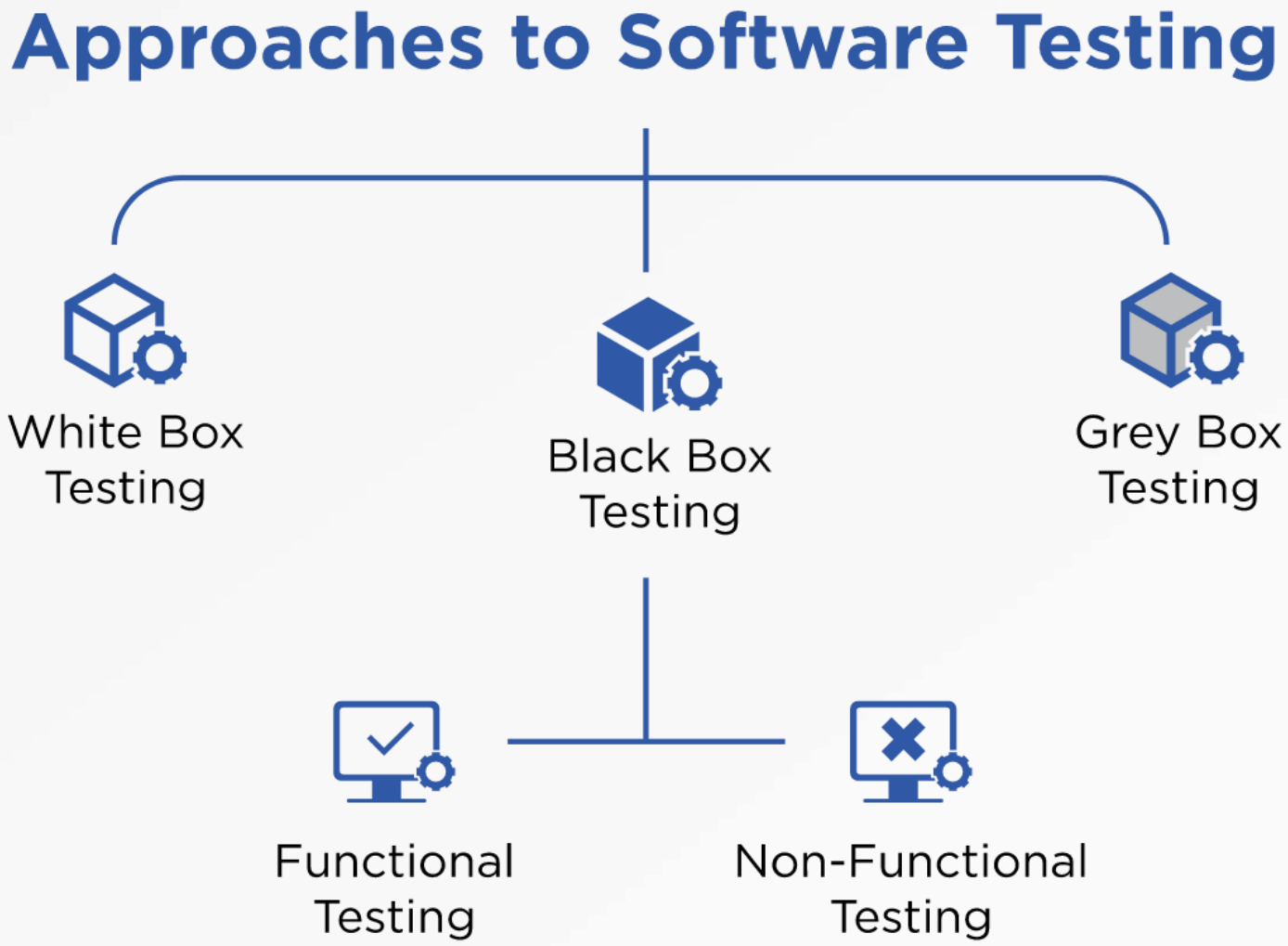

https://www.geeksforgeeks.org/introduction-of-software-design-process-set-2/

# Coding

- The coding is the process of transforming the design of a system into a computer language format.



# Testing


Approaches to Software Testing
White Box Testing
Black Box Testing
Grey Box Testing
Functional Testing
Non-Functional Testing

# Deployment



# Maintenance

- Systems evolve after deployment.
- Continued improvements and changes to your systems as needed.



# Need of well-defined Software Development

Following a well-defined methodology allows development teams to:

- Produce stable systems,
- Ensure customers are informed,
- Have a clear understanding of the task at hand,
- Offer better estimates,
- Identify potential pitfalls early on in the project.

# Software Development Life Cycle Models

- A software development life cycle (SDLC) refers to the various stages involved in system development.
- It may be focused on software, hardware, or a combination of both.

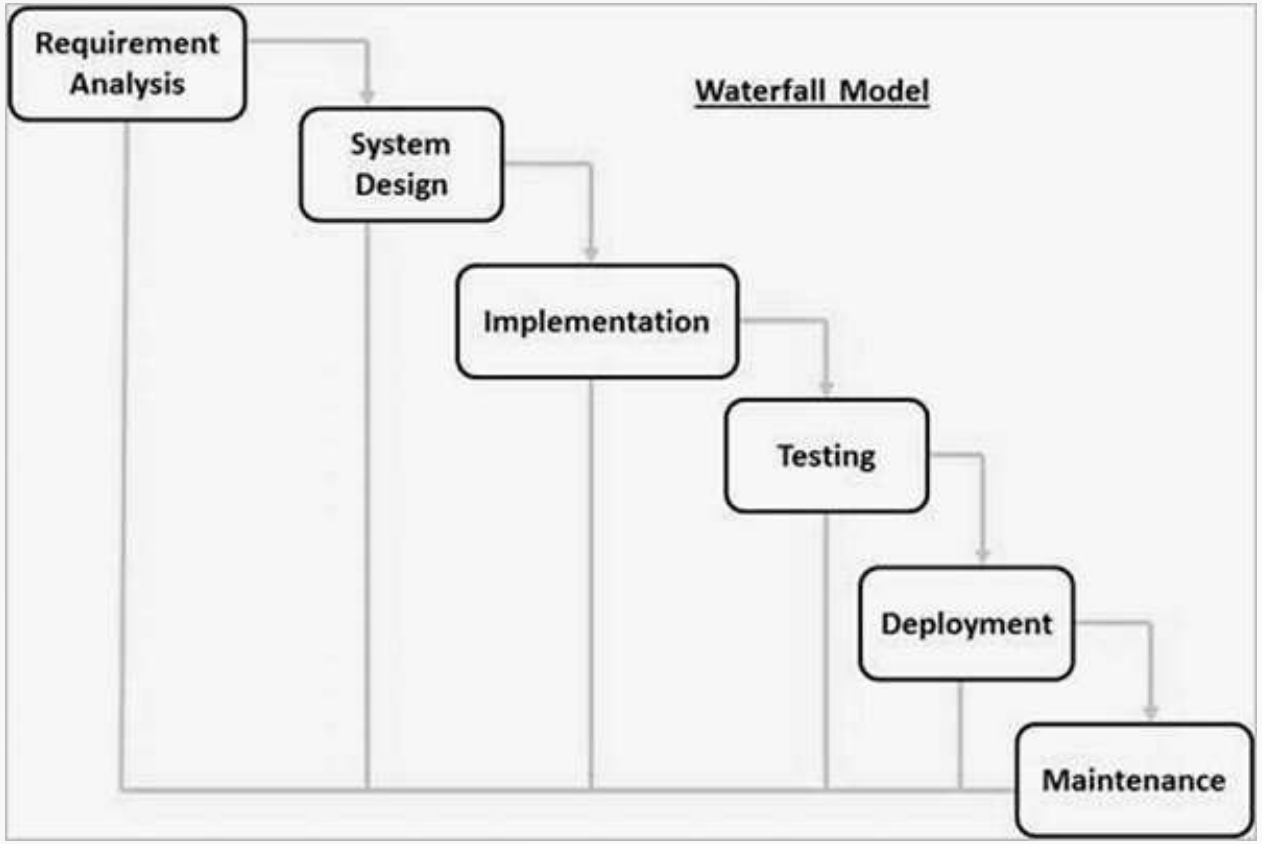
Most commonly used software development models are:

- Waterfall model
- Iterative Process models
- Agile Methodologies
- Evolutionary Prototyping

# Waterfall Model

- Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project.
- In "The Waterfall" approach, the whole process of software development is divided into separate phases.
- In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

# Waterfall Model



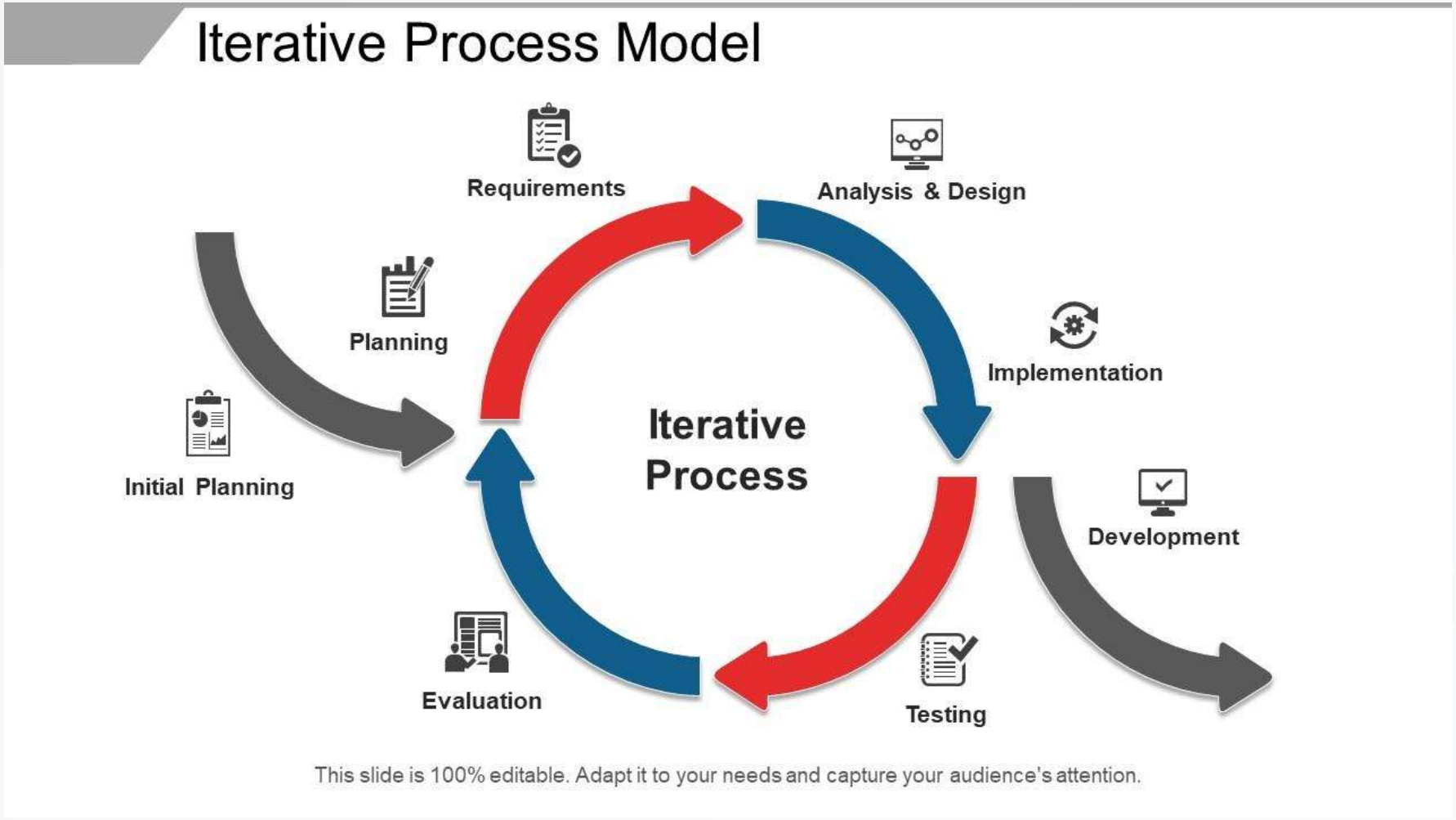
# Waterfall Model advantages and disadvantages

- 

# Iterative Process models

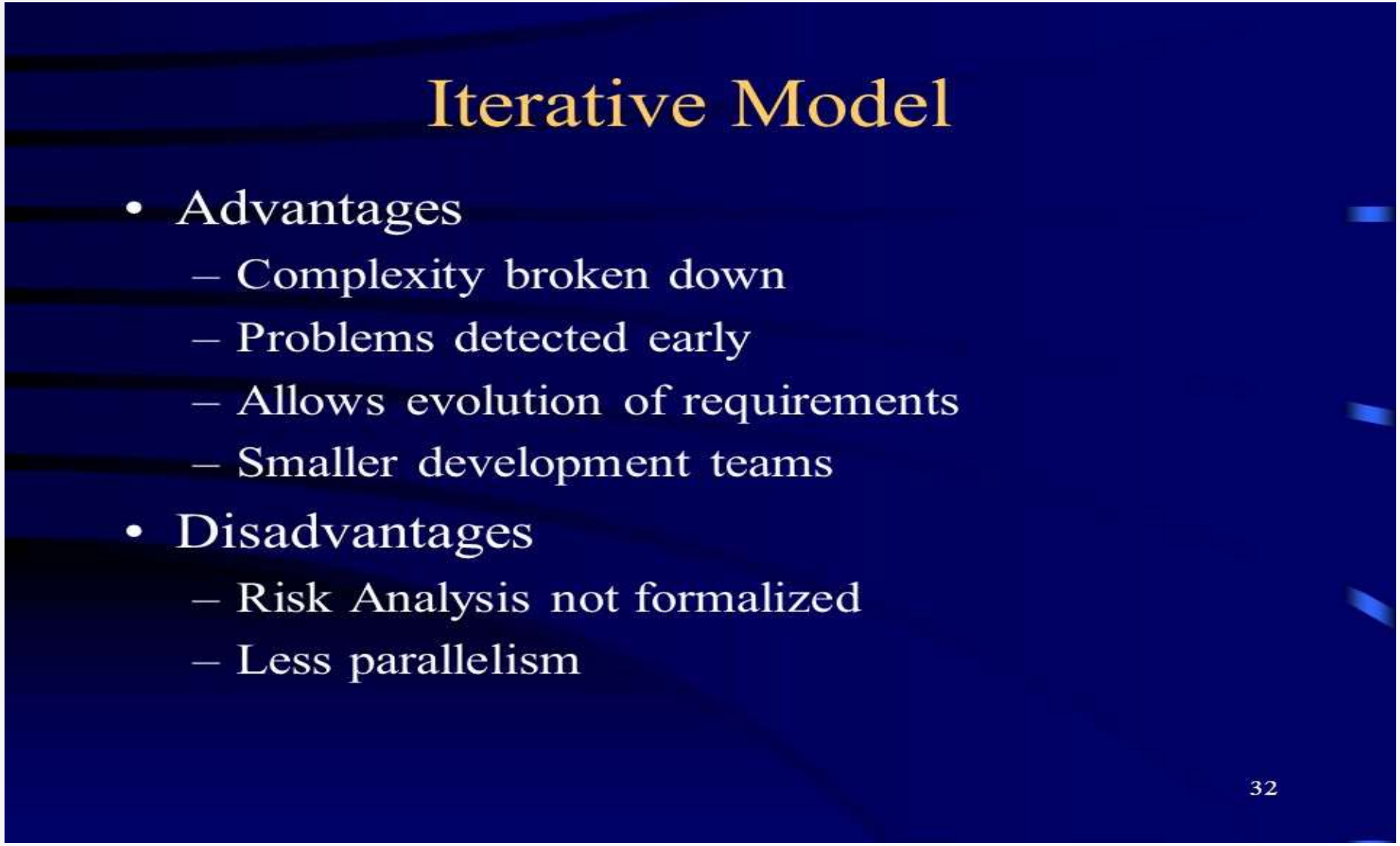
- In this Model, you can start with some of the software specifications and develop the first version of the software.
- After the first version if there is a need to change the software, then a new version of the software is created with a new iteration.
- Every release of the Iterative Model finishes in an exact and fixed period that is called iteration.
- The Iterative Model allows the accessing earlier phases, in which the variations made respectively.
- The final output of the project renewed at the end of the Software Development Life Cycle (SDLC) process.

# Iterative Process models




https://www.slideteam.net/iterative-process-model.html

# Iterative Process models advantages and disadvantages



# Iterative Process model - RUP

- Rational Unified Process-
  - Document Driven approach
  - An iterative and incremental approach
  - Extensive planning,
  - Codified Process,
  - Heavy Documentation ,
  - Big Design up Front

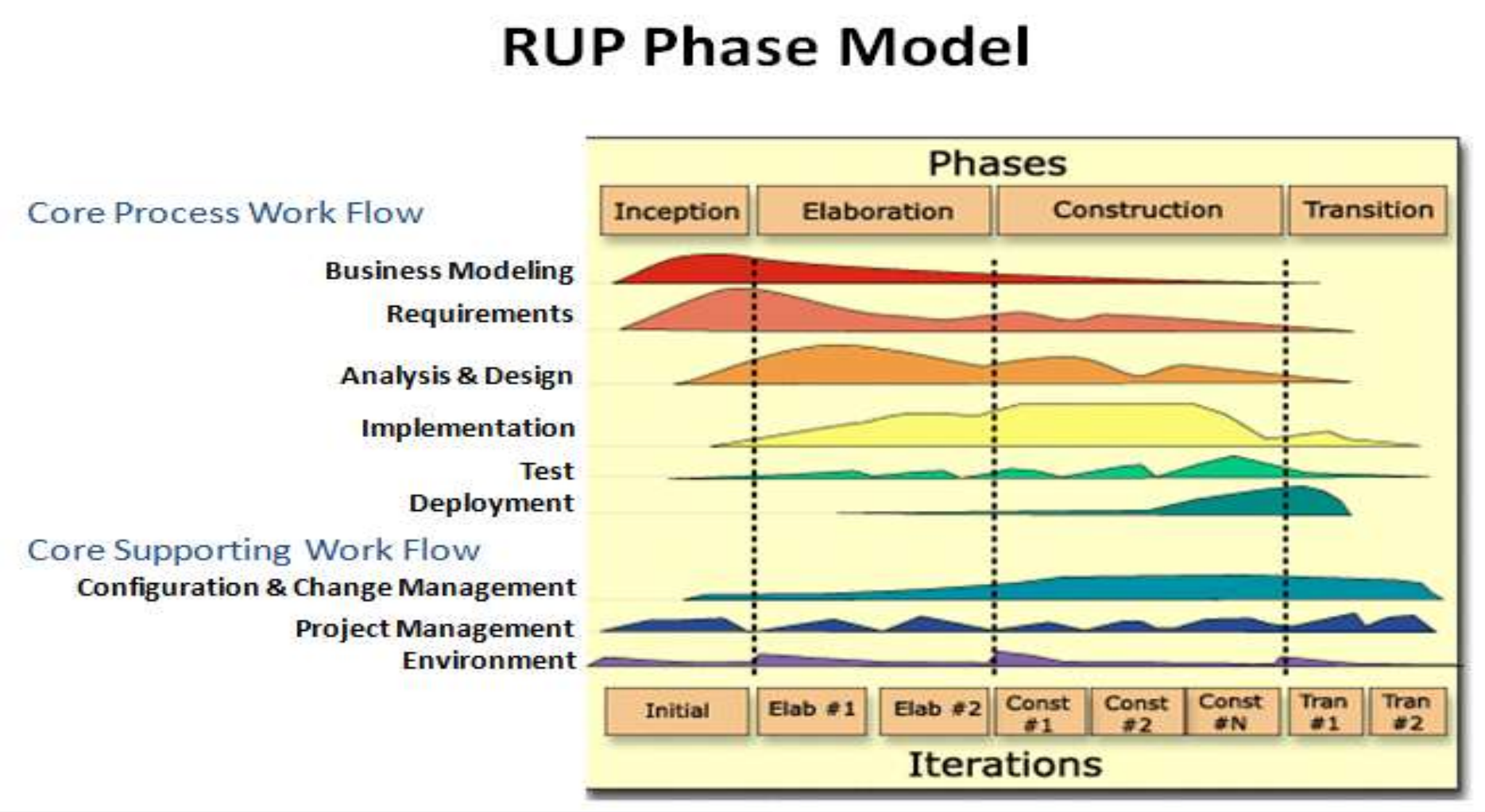
## RUP Phases:

Inception, Elaboration, Construction and Transition

## RUP Disciplines:

Main disciplines and Support disciplines

# Iterative Process model - RUP




https://www.mindcrafts.ch/2014/03/12/rup-rational-unified-process/

# RUP Strenghts and Weeknesses

**Strengths:**

- Straightforward, methodical and structured nature, Predictability, stability and high assurance

**Weaknesses**

- Slow adaptation to rapidly changing business requirements
- A tendency to be over budget
- A tendency to behind schedule
- Failed to provide dramatic progress in productivity, simplicity and reliability

# Agile Methodologies

- The meaning of Agile is swift or versatile."Agile process model" refers to a software development approach based on iterative development.
- Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning.
- Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks.

# Agile Manifesto

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

# Agile Software Development Characteristics

- Modularity:
- Iterative:
- Adaptive:
- Incremental and Risk Proactive:
- People Oriented:
- Collaborative:

# Referrences


- https://www.ibm.com/topics/software-development
- https://blog.bydrec.com/key-benefits-of-the-software-development-life-cycle
- https://www.javatpoint.com/software-engineering-iterative-model
- https://www.teach-ict.com/as_a2_ict_new/ocr/A2_G063/331_systems_cycle/prototyping_RAD/miniweb/pg3.htm
- https://www.geeksforgeeks.org/overview-of-plan-driven-development-pdd/

# Referrences


- https://slidemodel.com/requirements-gathering-project-management/
- https://www.educba.com/software-system-architecture/
- https://www.geeksforgeeks.org/introduction-of-software-design-process-set-2/
- https://www.javatpoint.com/software-engineering-coding
- https://www.leewayhertz.com/software-testing-process/
- https://www.mcgilly.com/services/deployment-and-maintenance/

# Software Construction & Development

Week-3

# Agile Methodologies




*Fig. Agile Model*

# Phases of Agile Model:

1. Requirements gathering:
2. Design the requirements:
3. Construction/ iteration:
4. Testing:
5. Deployment:
6. Feedback:

# Agile Methods:

- Scrum
- Crystal
- Dynamic Software Development Method(DSDM)
- Feature Driven Development(FDD)
- Lean Software Development
- eXtreme Programming(XP)

# When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

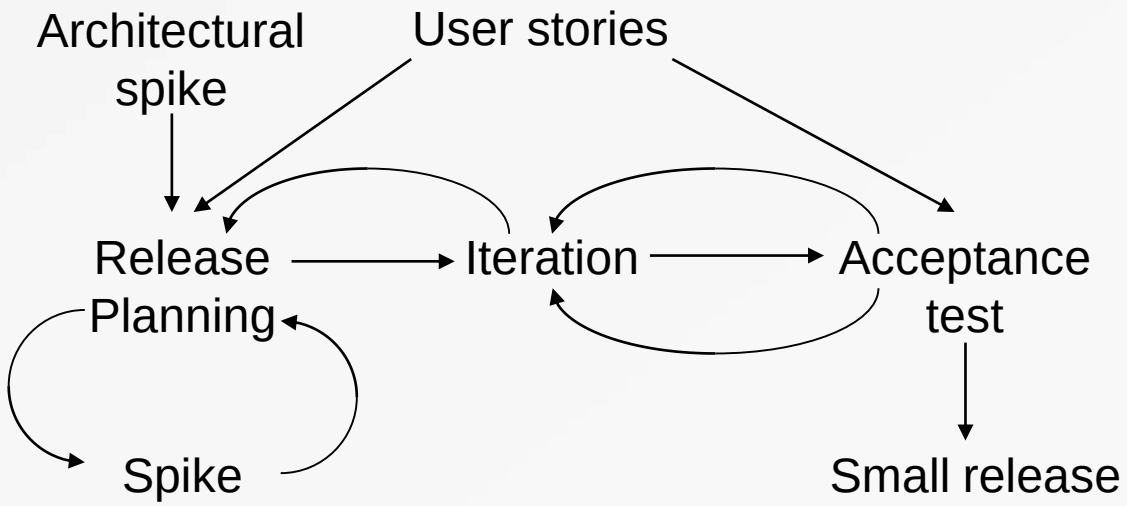
# Advantage(Pros) of Agile Method:

- Frequent Delivery
- Face-to-Face Communication with clients.
- Efficient design and fulfils the business requirement.
- Anytime changes are acceptable.
- It reduces total development time.

# Extreme Programming (XP) characterisitcs:

- Iterative and Incremental development,
- Customer collaboration,
- Frequent Delivery,
- Light and fast development,
- Light documentation

# Extreme Programming Working:

- 

# XP Practices:

12 Practices

- The Planning Game:
- Small Releases:
- Metaphor:
- Simple Design:
- Tests:
- Refactoring:

# XP Practices:

- Pair Programming:
- Collective Ownership:
- Continuous Integration:
- 40-hour Week:
- On-site customer:
- Coding Standard:

# XP Strengths:

- More customer-centric approach
- Welcome change in requirements
- More satisfactory relationship with the client
- Dramatic progress in productivity, simplicity and reliability
- Best engineering practices
- Tendency to be with in budget and as per schedule
- Built highest priority first .
- Lower software bugs rates

# XP Weeknesses:

- Not suitable for medium, complex and large scale projects.
- XP is not suitable in case of stable requirements.
- XP mainly focuses on people rather than processes
- XP doesn’t focus to model a well defined mechanism for business modeling.
- Not a well-known approach to handle non-functional requirements.
- XP doesn’t produce good quality results with new technology. XP doesn’t offer to capture user requirements through different elicitation techniques.

# Scrum Development Methodology:

- Short development cycle
- Higher customer satisfaction
- Low bug rates
- Quick adaptation to rapidly changing requirements
- Highest Priority Work
- Constant Feedback
- Control over Cost and Schedule

# Scrum basic Concepts:

- Scrum is another popular agile technique
- Sprint:
- The development cycle in Scrum is called Sprint which lasts for approximately 30 days. In Scrum there are two backlogs; Product Backlog: “All work to be performed in the foreseeable future, both well defined and requiring further definition”
- Sprint Backlog: “That work that is well-enough defined that it can be worked on with relatively little change over a period of 30 days or less and will result in a tangible, incremental deliverable”.

# Scrum Roles and Responsibilites:

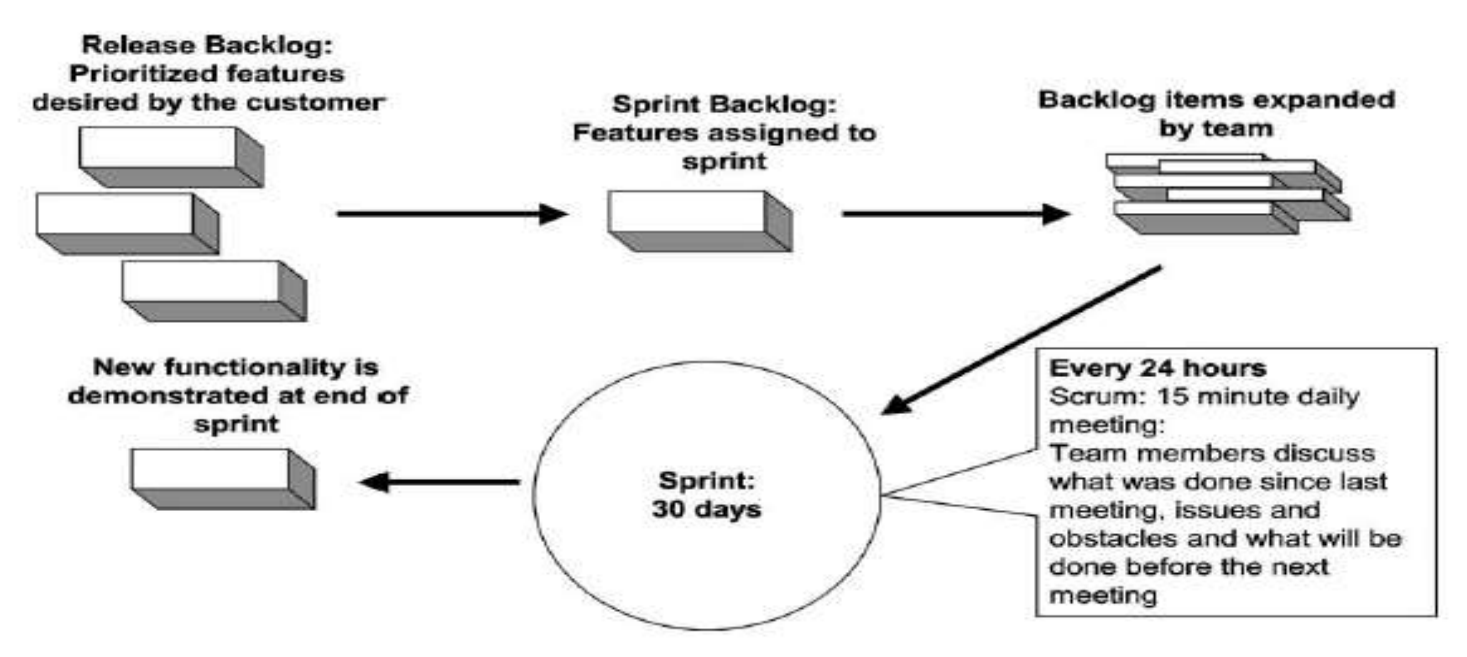
## Product Owner

- Responsibility is to define business value, requirements of project and Prioritization.
- Scrum Master
- Scrum Master is the Project Manager of the project. Scrum Master has to lead the project and make sure the software should meet the requirements set by the product owner
- Scrum Development Team:
- The responsibility of the Team is to meet the goal as well as the deadline. The team is dedicated to one project only.

lifecycle of a Sprint (iteration):

- Pre-Sprint Planning
- The Sprint Planning Meeting
- The Sprint Review Meeting
- The Sprint Retrospective
- A quality assurance practice
- The Daily Scrum Meeting

# Scrum Working:

- 

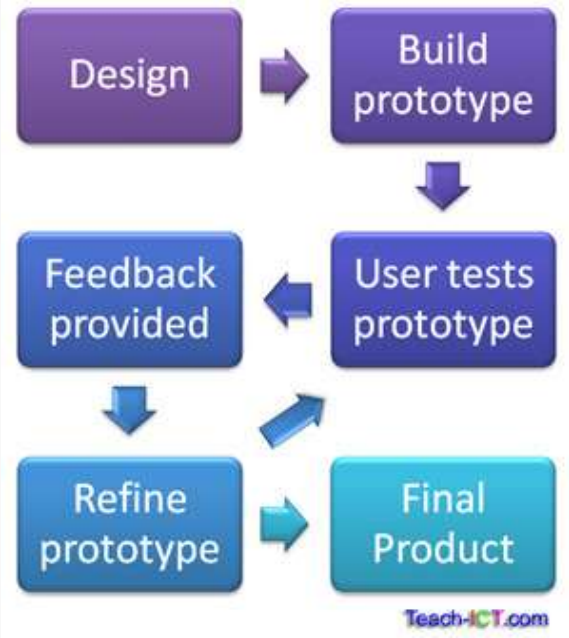

David Cohen. et Al. (2003)

Scrum Roles

# Evolutionary Prototyping

- The idea behind this is that an initial prototype is presented to the user.
- They provide feedback and suggestions for improvements.
- These are actioned by the developer who then presents a more refined prototype.
- The user once more provides feedback. The process is repeated.
- So at each stage the prototype 'evolves' towards the final system.
- Hence the term 'evolutionary prototyping'.

# Evolutionary Prototyping



# Risk-with Plan Driven Models

- Disadvantages of Plan Driven Development:

- Can not lodge changes any time.
- Longer distance in each repetition or increase.
- The shoulder that, future variations will not occur.
- Lack of user participation through the life cycle of the creation.
- Expensive for the dynamic expansion environment.

# Conventional Vs Agile

| | Software Process Model | Advantages | Disadvantages |
|---|---|---|---|
| **Plan Driven** | • Waterfall<br>• Incremental Development<br>• Iterative development<br>• Spiral Development<br>• Prototype Model<br>• Rapid Application Development | • Suitable for large systems and teams.<br>• Handles highly critical systems effectively.<br>• Appropriate for stable development environment.<br>• Require experienced personnel at the beginning.<br>• Success achieved through structure and order. | • Longer length in each iteration or increment.<br>• Cannot accommodate changes any time.<br>• Lack of user involvement throughout the life cycle of the product.<br>• Costly for the dynamic development environment.<br>• Assume that, future changes will not occur. |
| **Agile** | • Scrum model<br>• Extreme Programming (XP)<br>• Dynamic System Development Method<br>• Kanban<br>• Feature Driven Development | • Suitable for small to medium systems and teams.<br>• Can accommodate changes at any time.<br>• Effective for the dynamic development environment.<br>• Required expert agile personnel throughout the life cycle.<br>• Success achieved through freedom and chaos. | • Not suitable for large systems (except FDD).<br>• Shorter length in each iteration.<br>• Can accommodate changes at any time.<br>• Costly for the stable development environment.<br>• Assume that, frequent future changes will occur. |

https://www.researchgate.net/publication/329357135_ScrumFall_A_Hybrid_Software_Process_Model/figures?lo=1

# Referrences


- https://www.ibm.com/topics/software-development
- https://blog.bydrec.com/key-benefits-of-the-software-development-life-cycle
- https://www.javatpoint.com/software-engineering-iterative-model
- https://www.teach-ict.com/as_a2_ict_new/ocr/A2_G063/331_systems_cycle/prototyping_RAD/miniweb/pg3.htm
- https://www.geeksforgeeks.org/overview-of-plan-driven-development-pdd/

# Referrences


- https://slidemodel.com/requirements-gathering-project-management/
- https://www.educba.com/software-system-architecture/
- https://www.geeksforgeeks.org/introduction-of-software-design-process-set-2/
- https://www.javatpoint.com/software-engineering-coding
- https://www.leewayhertz.com/software-testing-process/
- https://www.mcgilly.com/services/deployment-and-maintenance/

# User interface design

- Designing effective interfaces for software systems

# Software GUI Construction

To suggest some general design principles for user interface design

- To explain different interaction styles
- To introduce styles of information presentation
- To describe the user support which should be built-in to user interfaces
- To introduce usability attributes and system approaches to system evaluation

# The user interface

- System users often judge a system by its interface rather than its functionality
- A poorly designed interface can cause a user to make catastrophic errors
- Poor user interface design is the reason why so many software systems are never used

# Graphical user interfaces

- Most users of business systems interact with these systems through graphical interfaces although, in some cases, legacy text-based interfaces are still used

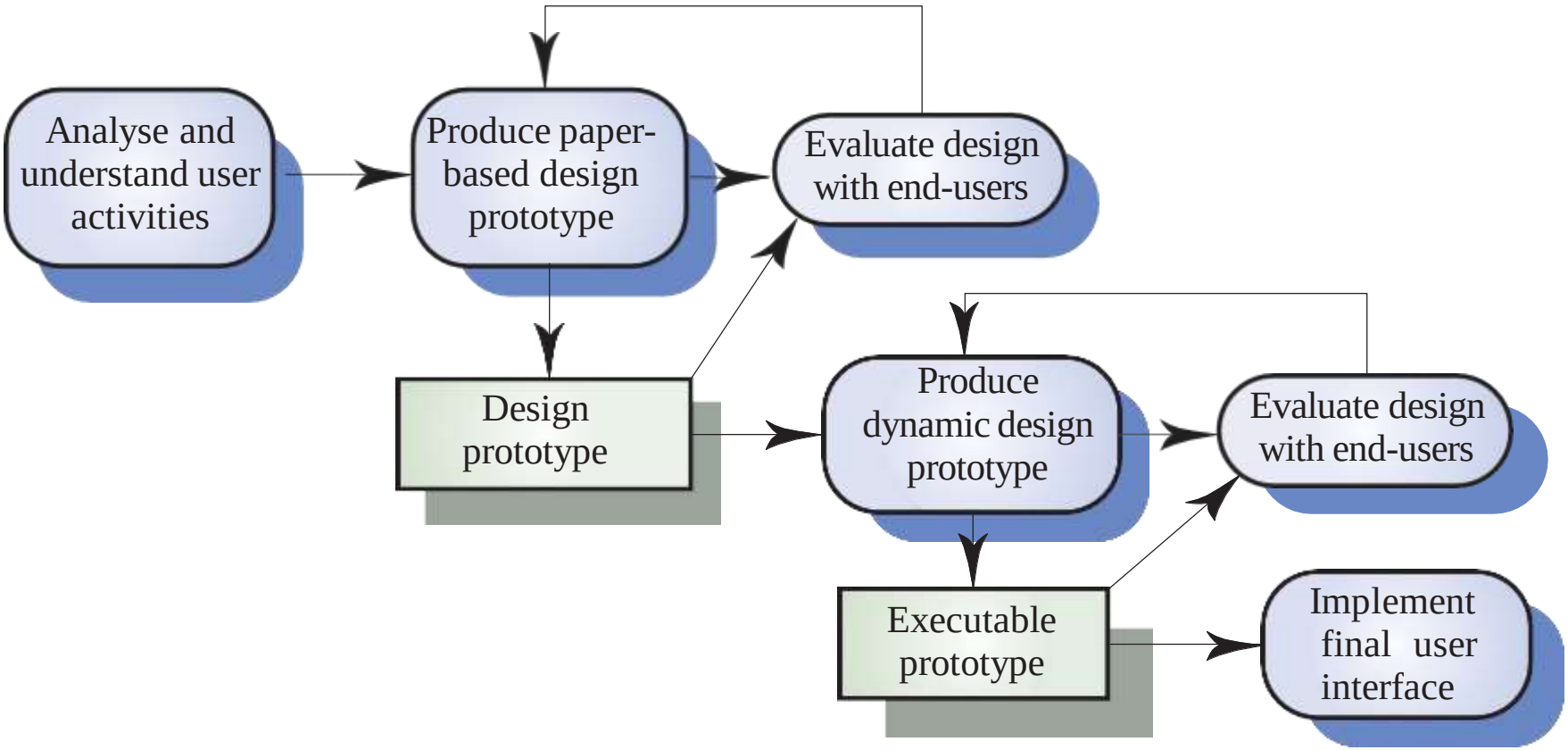
# GUI characteristics

| Characteristic | Description |
|---|---|
| Windows | Multiple windows allow different information to be displayed simultaneously on the user's screen. |
| Icons | Icons different types of information. On some systems, icons represent files; on others, icons represent processes. |
| Menus | Commands are selected from a menu rather than typed in a command language. |
| Pointing | A pointing device such as a mouse is used for selecting choices from a menu or indicating items of interest in a window. |
| Graphics | Graphical elements can be mixed with text on the same display. |

# GUI advantages

- They are easy to learn and use.
  - Users without experience can learn to use the system quickly.
- The user may switch quickly from one task to another and can interact with several different applications.
  - Information remains visible in its own window when attention is switched.
- Fast, full-screen interaction is possible with immediate access to anywhere on the screen

# User interface design process



# UI design principles

- UI design must take account of the needs, experience and capabilities of the system users
- Designers should be aware of people's physical and mental limitations (e.g. limited short-term memory) and should recognise that people make mistakes
- UI design principles underlie interface designs although not all principles are applicable to all designs

# User interface design principles

| Principle | Description |
|---|---|
| User familiarity | The interface should use terms and concepts which are drawn from the experience of the people who will make most use of the system. |
| Consistency | The interface should be consistent in that, wherever possible, comparable operations should be activated in the same way. |
| Minimal surprise | Users should never be surprised by the behaviour of a system. |
| Recoverability | The interface should include mechanisms to allow users to recover from errors. |
| User guidance | The interface should provide meaningful feedback when errors occur and provide context-sensitive user help facilities. |
| User diversity | The interface should provide appropriate interaction facilities for different types of system user. |

# Design principles

- User familiarity
  - The interface should be based on user-oriented terms and concepts rather than computer concepts. For example, an office system should use concepts such as letters, documents, folders etc. rather than directories, file identifiers, etc.
- Consistency
  - The system should display an appropriate level of consistency. Commands and menus should have the same format, command punctuation should be similar, etc.
- Minimal surprise
  - If a command operates in a known way, the user should be able to predict the operation of comparable commands

# Design principles

- Recoverability
  - The system should provide some resilience to user errors and allow the user to recover from errors. This might include an undo facility, confirmation of destructive actions, 'soft' deletes, etc.
- User guidance
  - Some user guidance such as help systems, on-line manuals, etc. should be supplied
- User diversity
  - Interaction facilities for different types of user should be supported. For example, some users have seeing difficulties and so larger text should be available

# User-system interaction

- Two problems must be addressed in interactive systems design
  - How should information from the user be provided to the computer system?
  - How should information from the computer system be presented to the user?
- User interaction and information presentation may be integrated through a coherent framework such as a user interface metaphor

# Interaction styles

- Direct manipulation
- Menu selection
- Form fill-in
- Command language
- Natural language

# Advantages and disadvantages

| Interaction style | Main advantages | Main disadvantages | Application examples |
|---|---|---|---|
| Direct manipulation | Fast and intuitive interaction<br>Easy to learn | May be hard to implement<br>Only suitable where there is a visual metaphor for tasks and objects | Video games<br>CAD systems |
| Menu selection | Avoids user error<br>Little typing required | Slow for experienced users<br>Can become complex if many menu options | Most general-purpose systems |
| Form fill-in | Simple data entry<br>Easy to learn | Takes up a lot of screen space | Stock control,<br>Personal loan processing |
| Command language | Powerful and flexible | Hard to learn<br>Poor error management | Operating systems,<br>Library information retrieval systems |
| Natural language | Accessible to casual users<br>Easily extended | Requires more typing<br>Natural language understanding systems are unreliable | Timetable systems<br>WWW information retrieval systems |

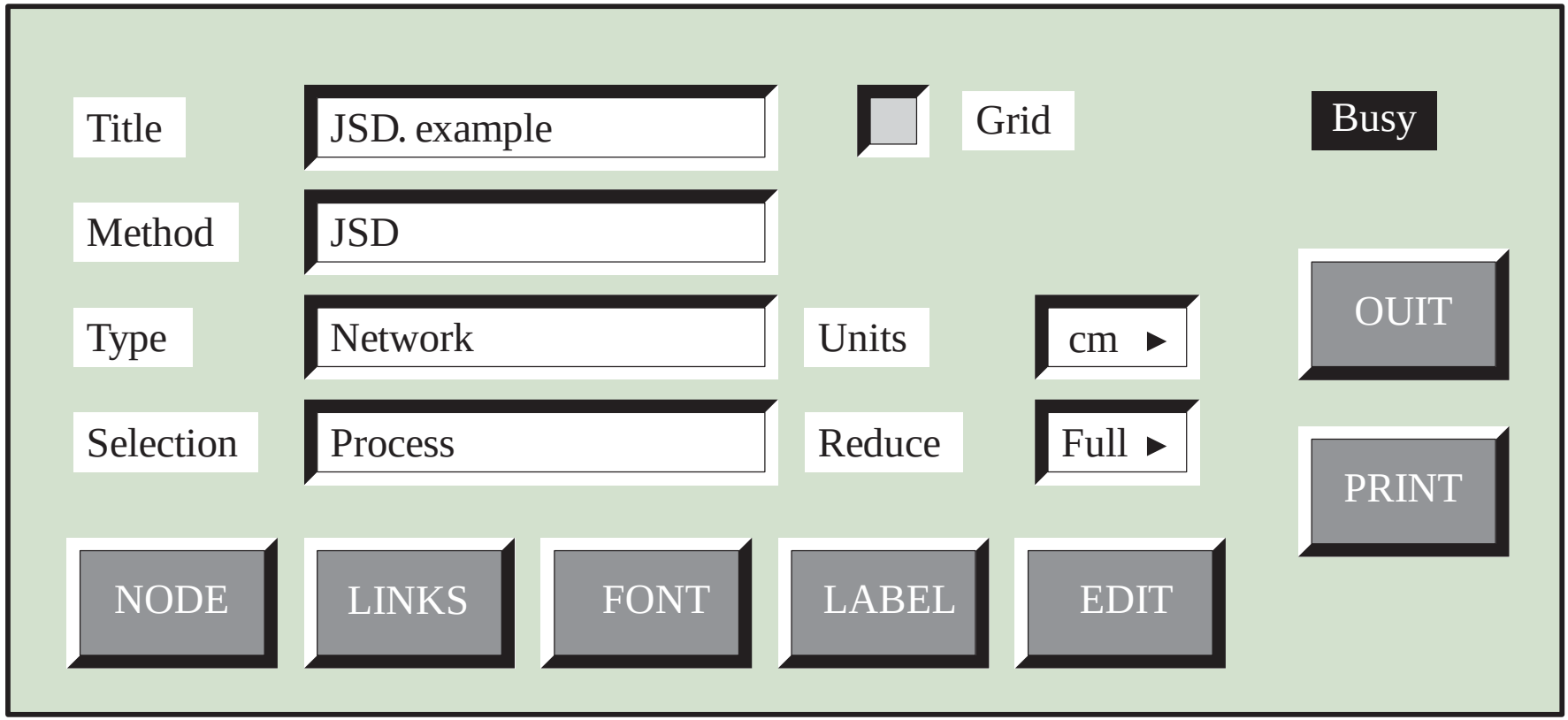
# Direct manipulation advantages

- Users feel in control of the computer and are less likely to be intimidated by it
- User learning time is relatively short
- Users get immediate feedback on their actions so mistakes can be quickly detected and corrected

# Direct manipulation problems

- The derivation of an appropriate information space model can be very difficult
- Given that users have a large information space, what facilities for navigating around that space should be provided?
- Direct manipulation interfaces can be complex to program and make heavy demands on the computer system

# Control panel interface



# Menu systems

- Users make a selection from a list of possibilities presented to them by the system
- The selection may be made by pointing and clicking with a mouse, using cursor keys or by typing the name of the selection
- May make use of simple-to-use terminals such as touchscreens

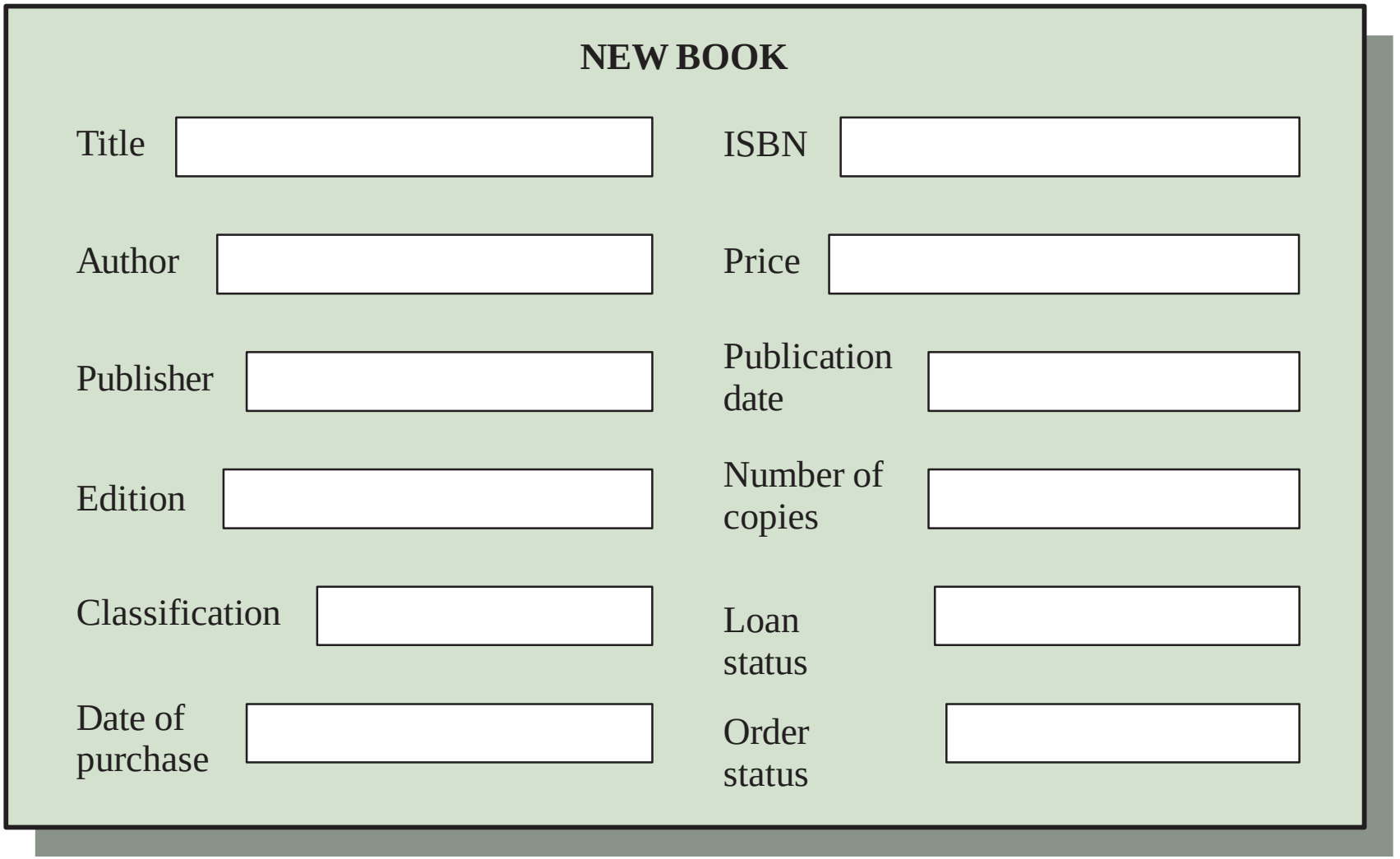
# Advantages of menu systems

- Users need not remember command names as they are always presented with a list of valid commands
- Typing effort is minimal
- User errors are trapped by the interface
- Context-dependent help can be provided. The user's context is indicated by the current menu selection

# Problems with menu systems

- Actions which involve logical conjunction (and) or disjunction (or) are awkward to represent
- Menu systems are best suited to presenting a small number of choices. If there are many choices, some menu structuring facility must be used
- Experienced users find menus slower than command language

# Form-based interface



# Command interfaces

- User types commands to give instructions to the system e.g. UNIX
- May be implemented using cheap terminals.
- Easy to process using compiler techniques
- Commands of arbitrary complexity can be created by command combination
- Concise interfaces requiring minimal typing can be created

# Problems with command interfaces

- Users have to learn and remember a command language. Command interfaces are therefore unsuitable for occasional users
- Users make errors in command. An error detection and recovery system is required
- System interaction is through a keyboard so typing ability is required

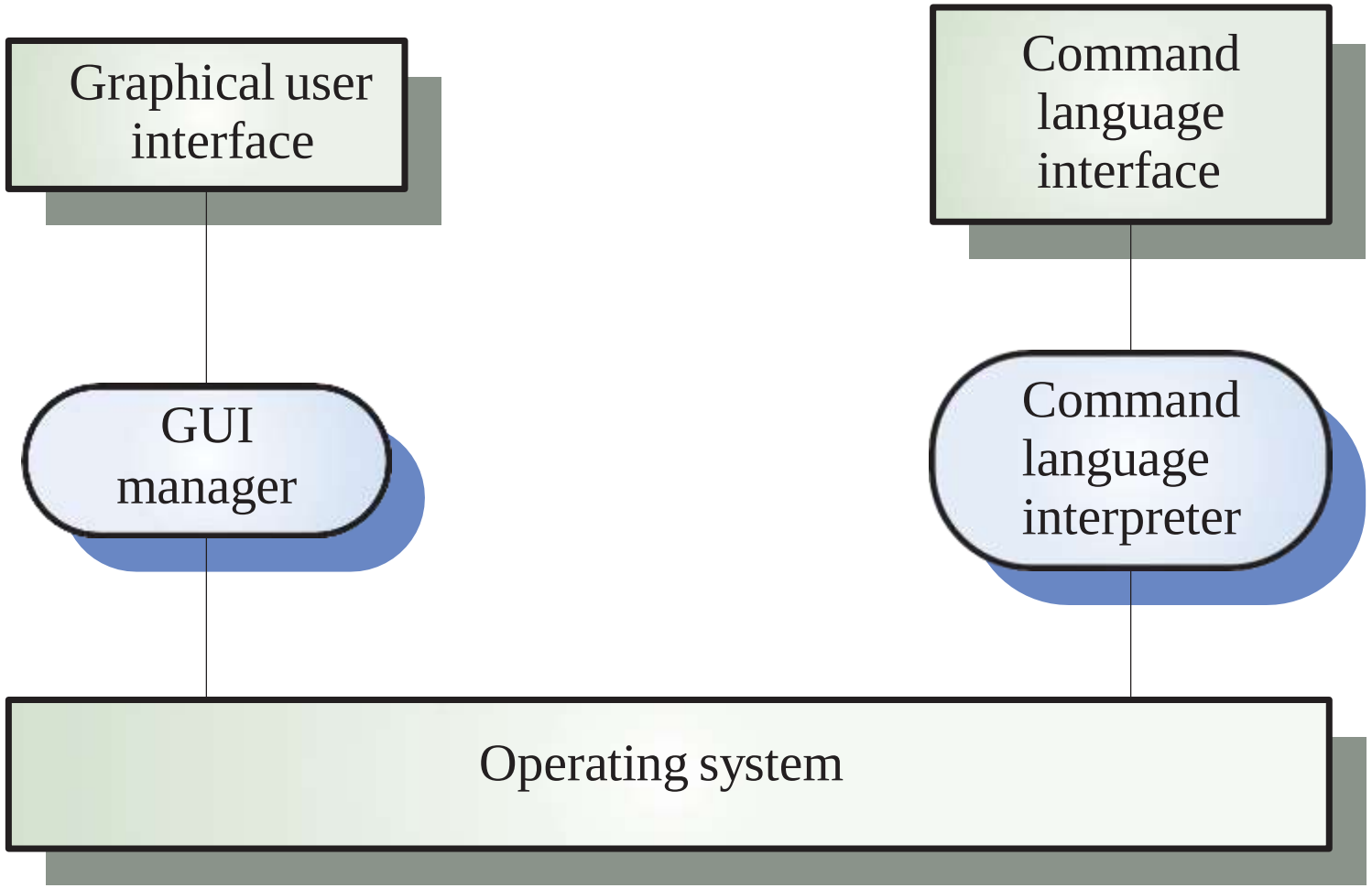
# Command languages

- Often preferred by experienced users because they allow for faster interaction with the system
- Not suitable for casual or inexperienced users
- May be provided as an alternative to menu commands (keyboard shortcuts). In some cases, a command language interface and a menu-based interface are supported at the same time

# Natural language interfaces

- The user types a command in a natural language. Generally, the vocabulary is limited and these systems are confined to specific application domains (e.g. timetable enquiries)
- NL processing technology is now good enough to make these interfaces effective for casual users but experienced users find that they require too much typing

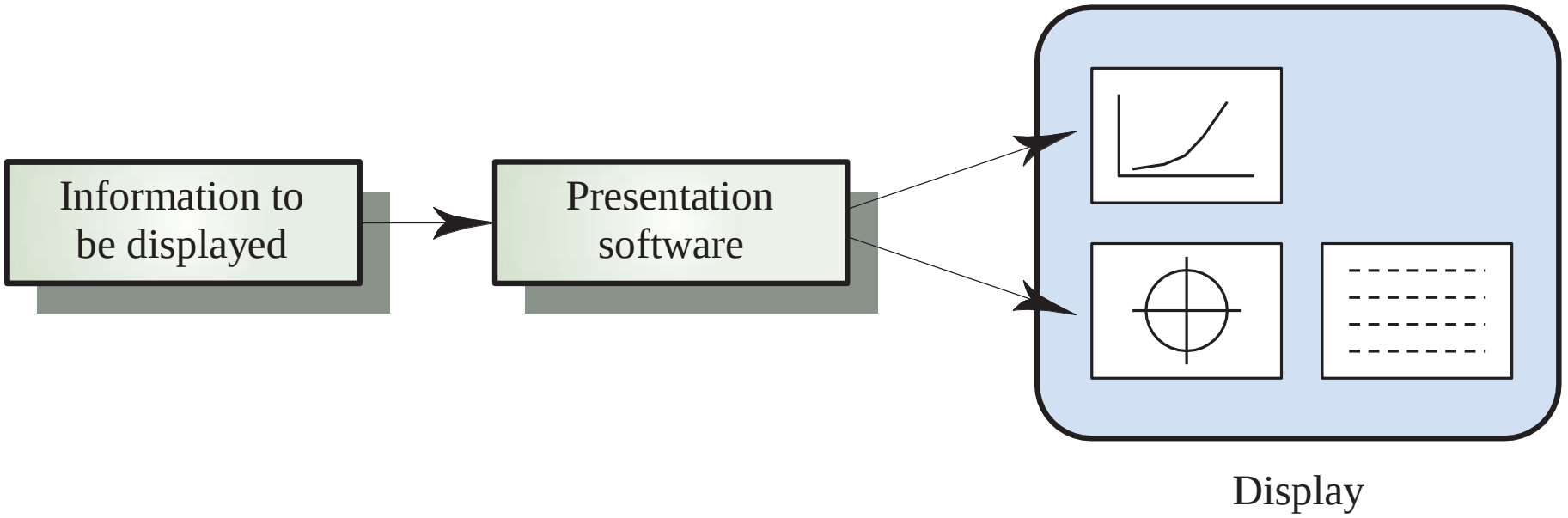
# Multiple user interfaces



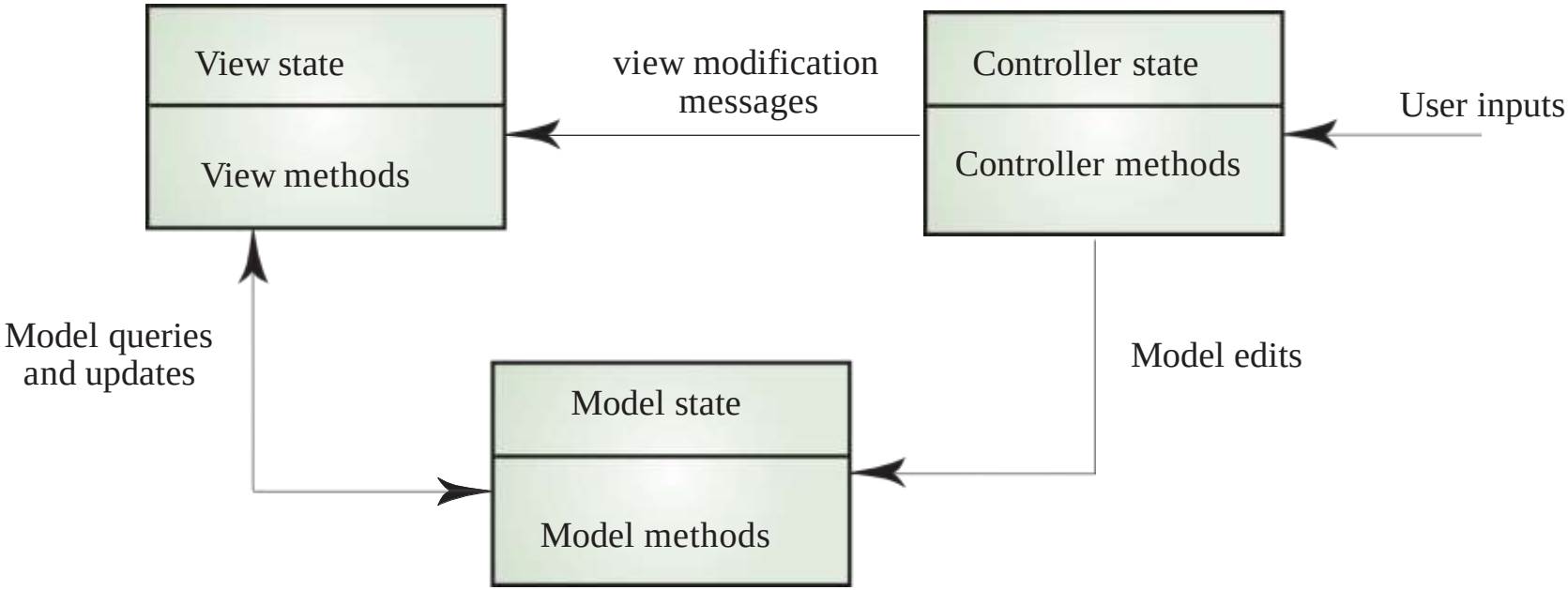
# Information presentation

- Information presentation is concerned with presenting system information to system users
- The information may be presented directly (e.g. text in a word processor) or may be transformed in some way for presentation (e.g. in some graphical form)
- The Model-View-Controller approach is a way of supporting multiple presentations of data

# Information presentation



# Model-view-controller



# Information presentation

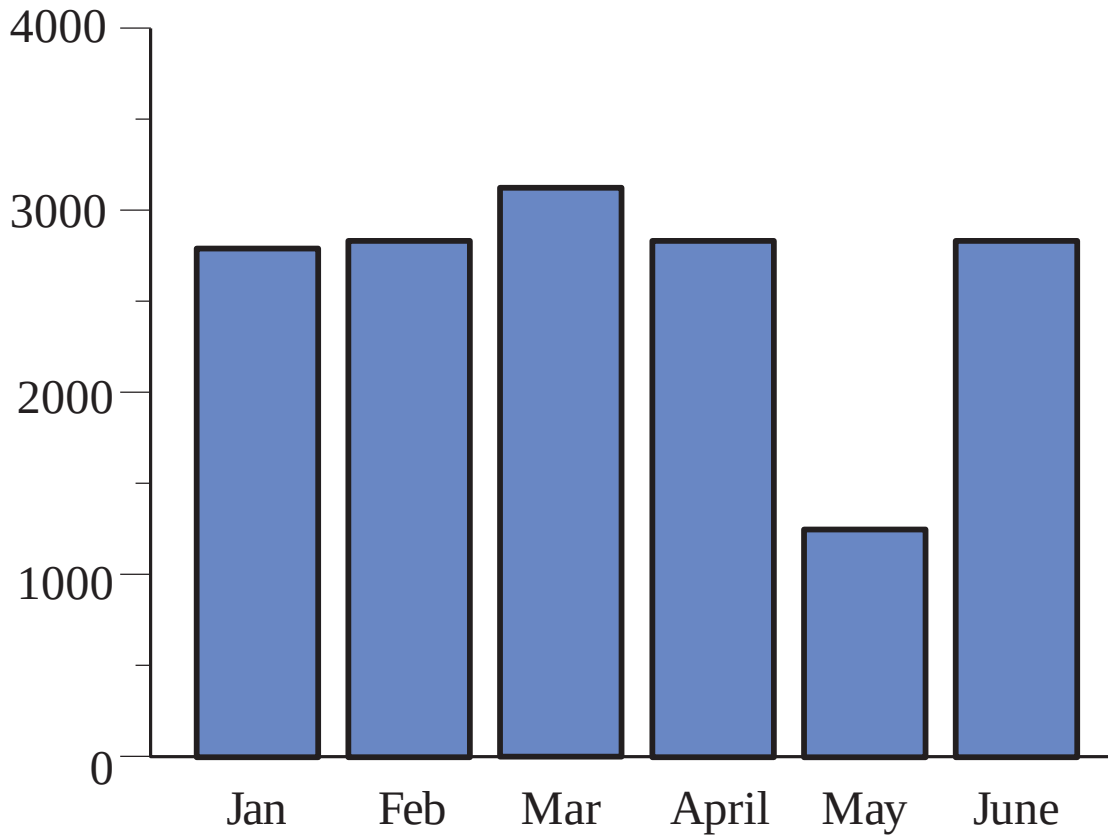
- Static information
  - Initialised at the beginning of a session. It does not change during the session
  - May be either numeric or textual
- Dynamic information
  - Changes during a session and the changes must be communicated to the system user
  - May be either numeric or textual

# Information display factors

- Is the user interested in precise information or data relationships?
- How quickly do information values change? Must the change be indicated immediately?
- Must the user take some action in response to a change?
- Is there a direct manipulation interface?
- Is the information textual or numeric? Are relative values important?

# Alternative information presentations

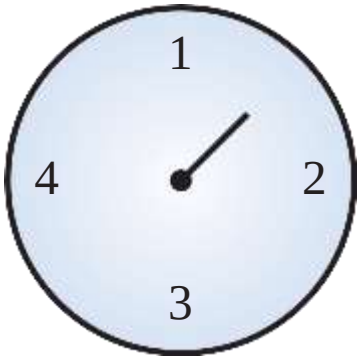
| Jan | Feb | Mar | April | May | June |
|---|---|---|---|---|---|
| 2842 | 2851 | 3164 | 2789 | 1273 | 2835 |



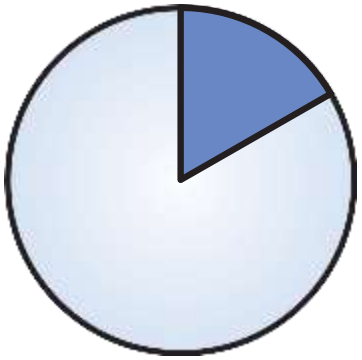
# Analogue vs. digital presentation

- Digital presentation
  - Compact - takes up little screen space
  - Precise values can be communicated
- Analogue presentation
  - Easier to get an 'at a glance' impression of a value
  - Possible to show relative values
  - Easier to see exceptional data values

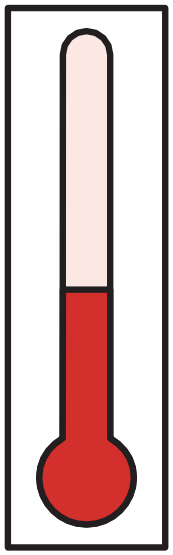
# Dynamic information display



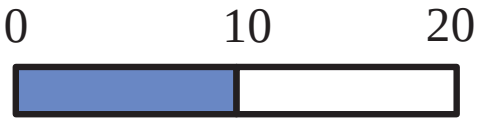

Dial with needle



Pie chart

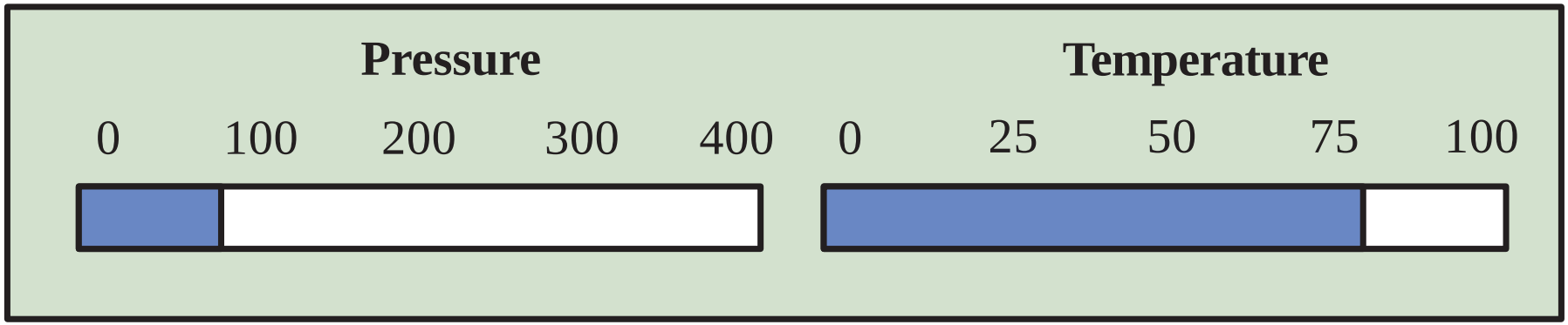


Thermometer

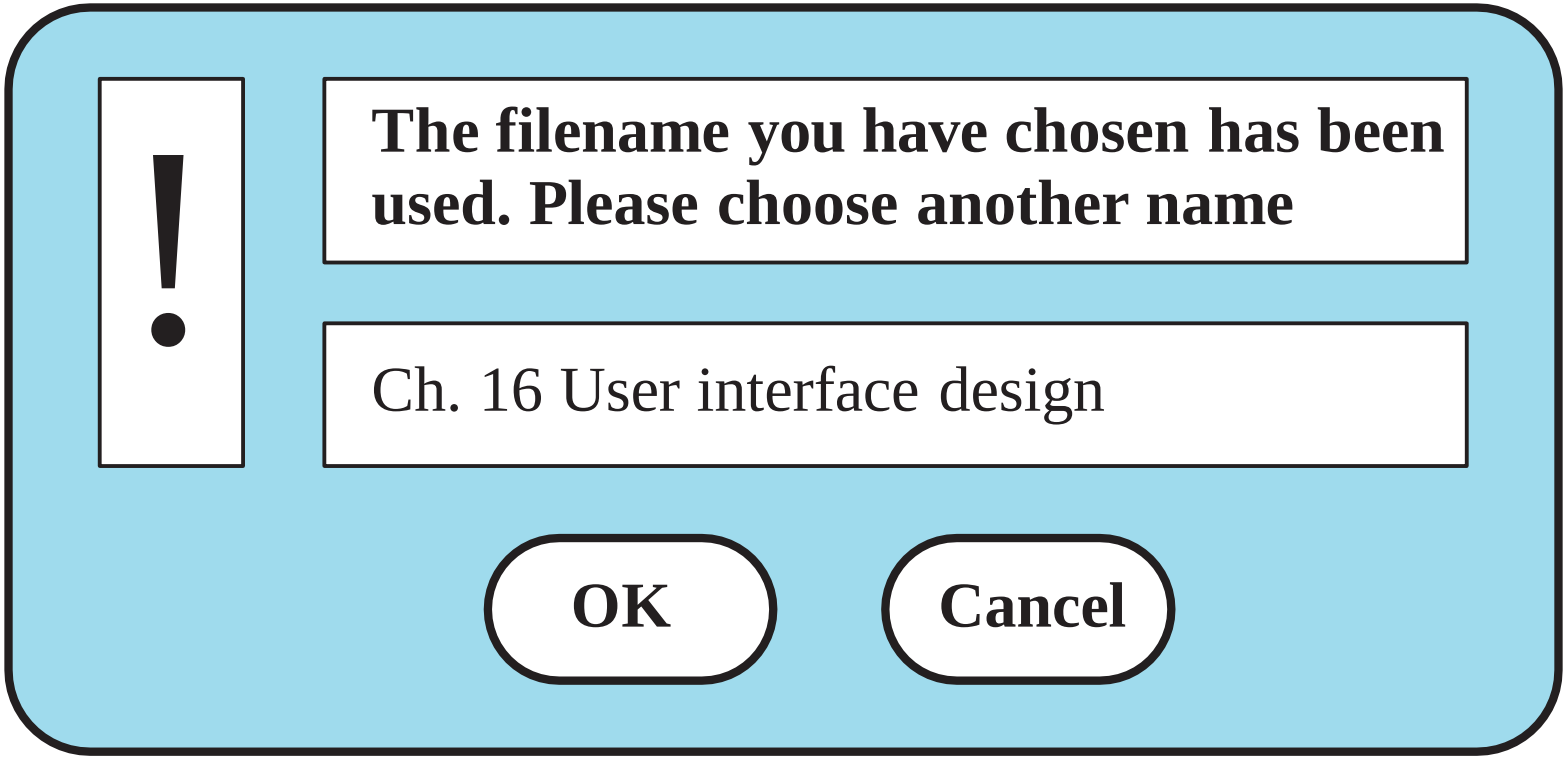



Horizontal bar

# Displaying relative values



# Textual highlighting



# Data visualisation

- Concerned with techniques for displaying large amounts of information
- Visualisation can reveal relationships between entities and trends in the data
- Possible data visualisations are:
  - Weather information collected from a number of sources
  - The state of a telephone network as a linked set of nodes
  - Chemical plant visualised by showing pressures and temperatures in a linked set of tanks and pipes
  - A model of a molecule displayed in 3 dimensions
  - Web pages displayed as a hyperbolic tree

# Colour displays

- Colour adds an extra dimension to an interface and can help the user understand complex information structures
- Can be used to highlight exceptional events
- Common mistakes in the use of colour in interface design include:
  - The use of colour to communicate meaning
  - Over-use of colour in the display

# Colour use guidelines

- Don't use too many colours
- Use colour coding to support use tasks
- Allow users to control colour coding
- Design for monochrome then add colour
- Use colour coding consistently
- Avoid colour pairings which clash
- Use colour change to show status change
- Be aware that colour displays are usually lower resolution

# User support

- User guidance covers all system facilities to support users including on-line help, error messages, manuals etc.
- The user guidance system should be integrated with the user interface to help users when they need information about the system or when they make some kind of error
- The help and message system should, if possible, be integrated

# What is usability



# Usability

**Definition :**

ISO 9241-210, defines usability as “extent to which a system, product or service can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use”.

**Usability Perspectives**

(i) A product-oriented perspective emphasizes that usability can be measured with regard to the ergonomic characteristics of a product.

(ii) A user-oriented perspective, usability is represented by means of the user's mental effort in product usage and his/her attitude towards the product.

(iii) A performance-oriented perspective, usability is described in terms of the interaction of the user with the product,

(iv) A context-oriented perspective emphasizes that usability is dependent on the user group that is studied, the tasks those users are performing, and the environment in which the tasks are completed.

# Usability Engineering

Usability is all about how users interact with technology
Usability engineering requires a firm knowledge of computer science and psychology and approaches product development based on customer feedback.
A usability engineer works hand-in-hand with customers, working to develop a better understanding of the functionality

# Why isn't UE done Regulalrly?

Developers believe

- it takes too long
- is too expensive
- is not critical to development
- they can afford to learn about user problems during "live" use

Also developers may not know how to do it

# Benefits of UE to an Organization

- Reduce training costs
- Reduce development costs
  - Identify and fix problems early
- Reduce support costs; minimize need for
  - support personnel/help desks
  - fixes, maintenance, upgrades
- Enhance organization's reputation - positive "word-of-mouth" trade
- Larger numbers of "hit" and "return visit" rates

# Benefits of UE to the User

Less time to complete work

Greater success with tasks

Increased user satisfaction

# Typical Web-site Usability Problems

- Navigation
  - Knowing where you are
  - Finding what you want
- Structure of web site
- Layout
  - Needs sufficient white space
  - Use of large graphics
- “The Google problem”

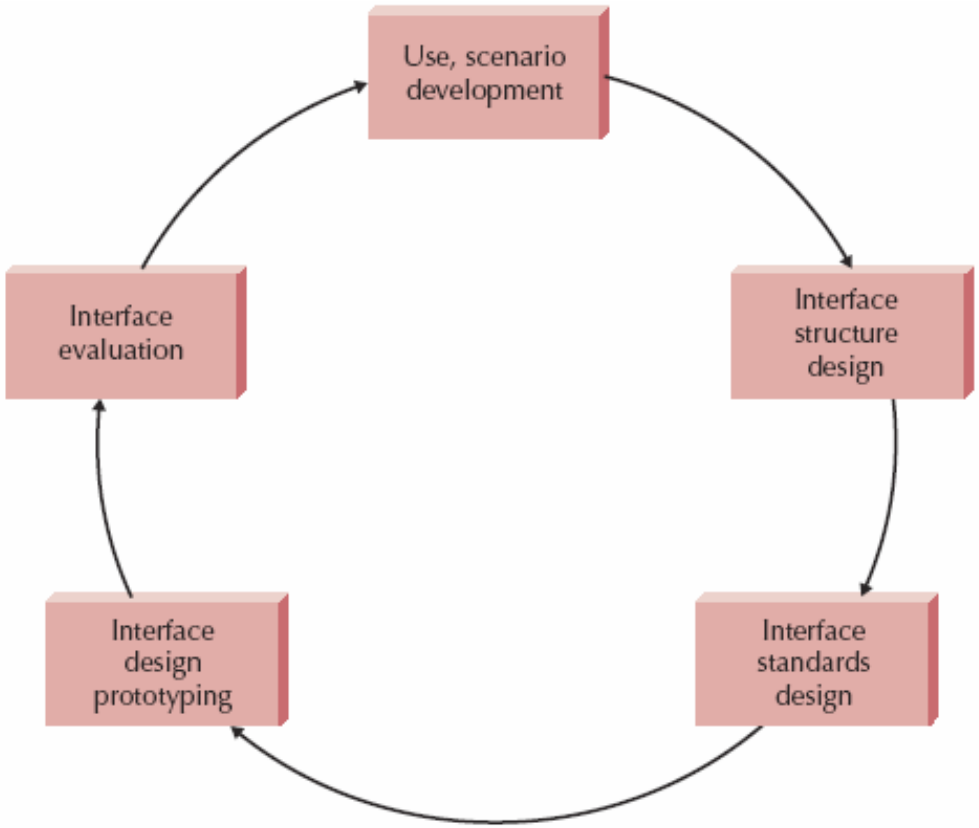
# User Centered Design

UCD is a dialog between the customer and the designer

Rules of thumb:

- Get to know and understand the users.
- Build an application, applying usability principles.
- Test designs by observing users in a real work setting (environment and work load).

# Usability design Process

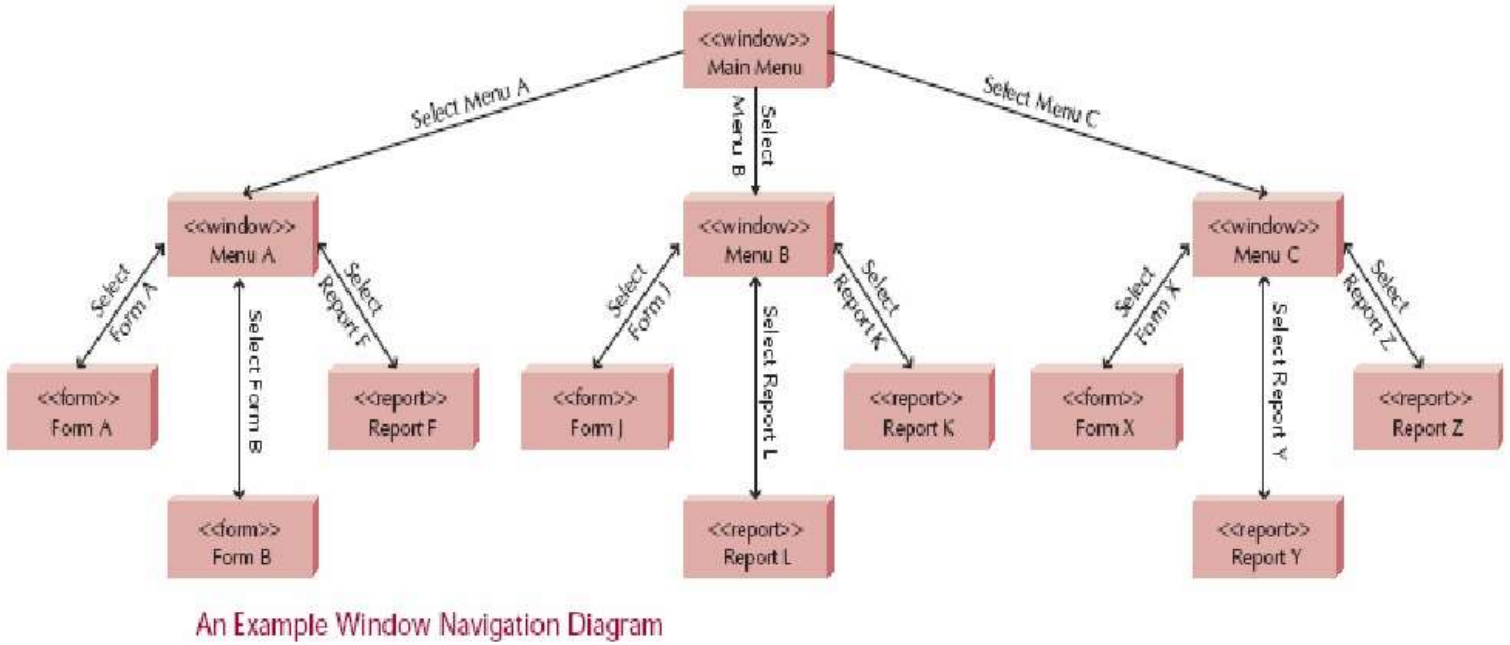


# Use scenario Development

You already know about use cases

Use scenarios are just a single thread through a use case – much simpler!

# UI Structure (architecture diagram of interface)




An Example Window Navigation Diagram

# Interface Standard Design (one video to another..what standard must follow)

## UI design principles: key rules and guidelines

1. Make it easy to explore and use
2. Give users control of the interface
3. Create a layout that works efficiently
4. Offer a consistent interface
5. Use real-world metaphors in your UI
6. Know your way around design materials
7. Start with black-and-white UI design, then build on it
8. Feedback and context is important
9. Fine-tune your wireframing game
10. Get familiar with user testing and the world of usability

# Interface Design Prototyping (develop storyboard on paper)

- Storyboard
- HTML Prototype
- Language Prototype
- Interface Evaluation
  - Heuristic
  - Walkthrough
  - Interactive
- Formal Usability Testing

# Usability Principles

| Principle | Description |
|---|---|
| Layout | The interface should be a series of areas on the screen that are used consistently for different purposes—for example, a top area for commands and navigation, a middle area for information to be input or output, and a bottom area for status information. |
| Content awareness | Users should always be aware of where they are in the system and what information is being displayed. |
| Aesthetics | Interfaces should be functional and inviting to users through careful use of white space, colors, and fonts. There is often a tradeoff between including enough white space to make the interface look pleasing without losing so much space that important information does not fit on the screen. |
| User experience | Although ease of use and ease of learning often lead to similar design decisions, there is sometimes a tradeoff between the two. Novice users or infrequent users of software will prefer ease of learning, whereas frequent users will prefer ease of use. |
| Consistency | Consistency in interface design enables users to predict what will happen before they perform a function. It is one of the most important elements in ease of learning, ease of use, and aesthetics. |
| Minimal user effort | The interface should be simple to use. Most designers plan on having no more than three mouse clicks from the starting menu until users perform work. |

# GUI Design is Multi Disciplinary

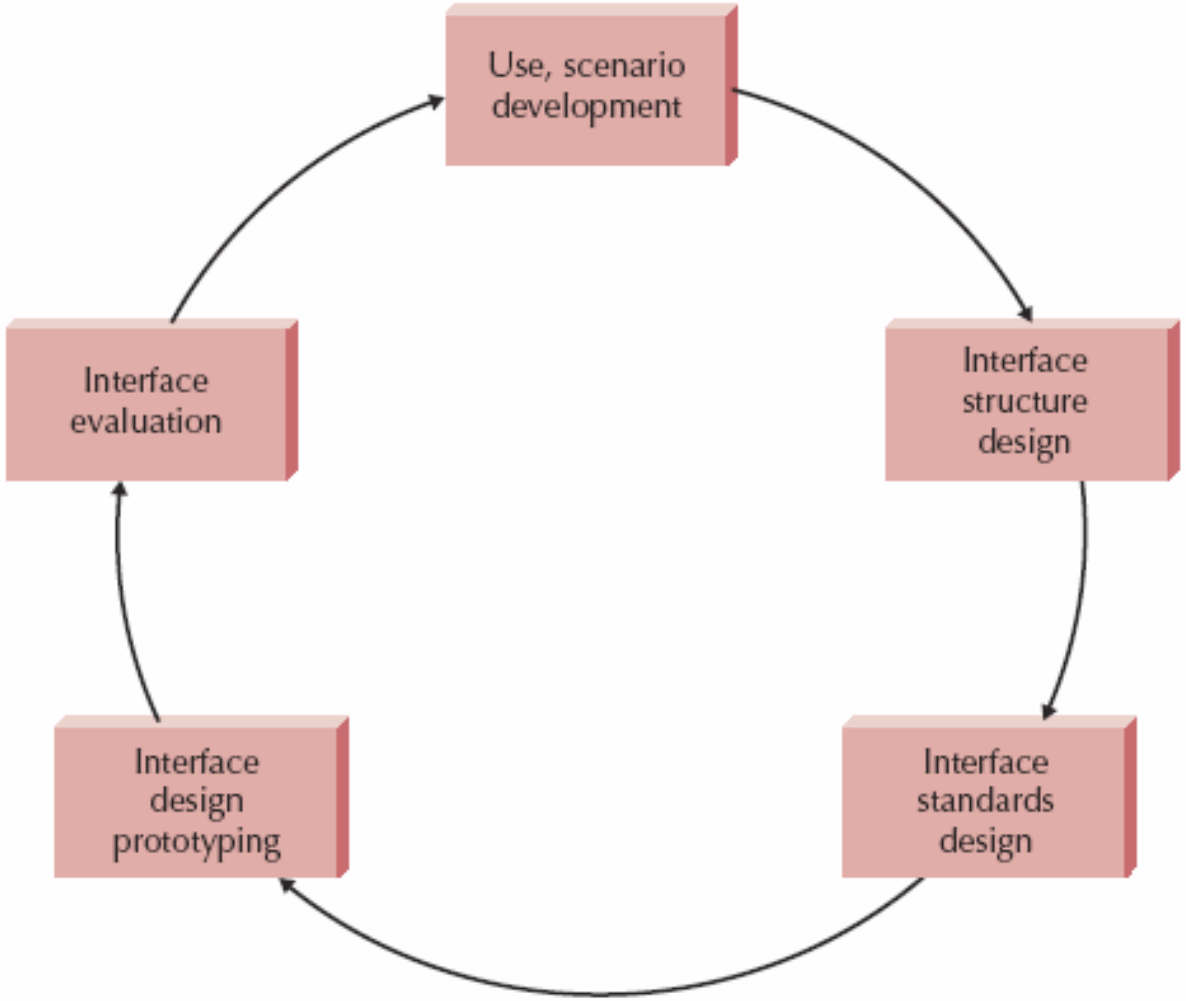
A team includes

- Analyst
- Designer
- Technology expert
- Graphic artist
- Social and behavioral scientist
- Programmer

# Software Construction and Development

Week-7

# Usability design Process



# Usability evaluation of Interfaces

- Definition
  - consists of methodologies for measuring the usability aspects of a system's user interface (UI) and identifying specific problems
- Usability Evaluation Methods
  - testing, inspection, and inquiry
- Approaches to evaluate usability
  - Summative
    - Real users perform the real tasks on some defined applications and empirically measure the usability in terms of effectiveness, efficiency and user satisfaction.
  - Formative
    - Formative approach employs the experts for UE using heuristics or guidelines

# Heuristics for Interface Usability Evaluation

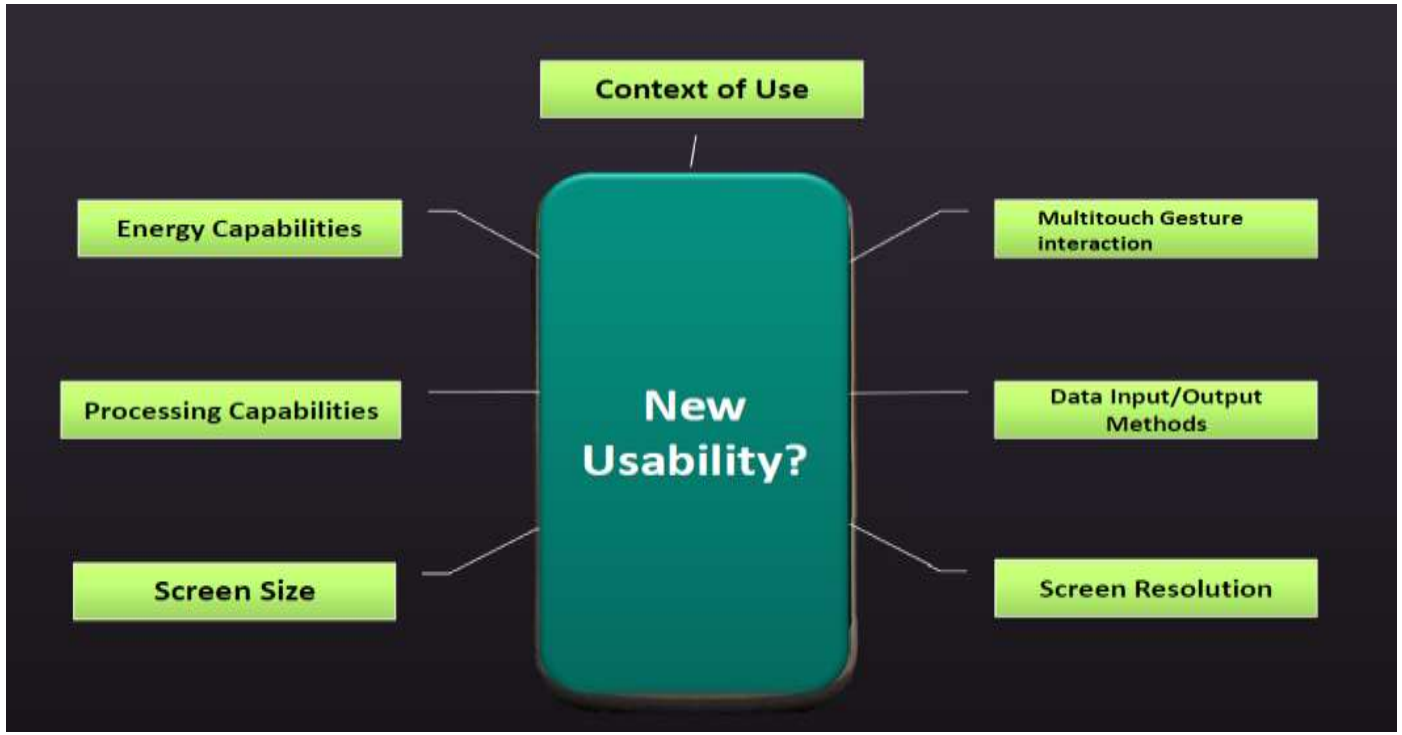
- An inspection based usability evaluation method
- Consists a systematic evaluation in which several experts—usually 3 to 5—analyze the interface of an interactive system and make observations based on a set of principles known as usability heuristics.
- The goal of heuristic evaluation is to discover problems related to the usability in the interface design

- Protocol of a typical heuristic evaluation involves that each evaluator spends time inspecting the system interface in an independent manner, making note of every usability problem found. Once all evaluators have complete the evaluation, their individual findings are joined in a unique common list of usability problems associated to those heuristics that were violated. The evaluators must rate the severity of each problem, according to a scale of 0 (lowest severity) to 4 (highest severity)
- Nielsen proposed 10 general usability heuristics for the design and evaluation of interfaces, Known as “Nielsen’s heuristics”.

# Design and Evaluation Heuristics for Smartphone : Case Study

- Smartphone Features with associated Usability Limitations:



# Smartphone Features with associated Usability Limitations:

| | |
|---|---|
| **Small screen size** | Small screen size may cause congestion of information on display area and the user can mistakenly click the incorrect option due to substantial information. On the other hand, if the mobile UI contain small amount of information, then, there are chances that the end user could not find some of the core information. |
| **Limited input/output capabilities** | Adapted gestures for interaction may change the value of mobile usability. Similarly, simulated keyboard also causes usability issues. Users concurrently spend the cognitive workload between the content and the keypad region. |
| **Limited computational resources** | User demands that smartphone should complete the task without any mistake and behave more likely a desktop. |
| **Limited power,** | More interface brightness, internet connectivity and working of multiple sensors requires more consumption of battery power. End user suffers and compromises in such cases. |
| **Types of tasks** | Mobile devices provide limited capability in it |
| **Display resolution,** | Comparatively, lower resolution of mobile resulting in lesser quality images. |
| **Mobile Physical buttons** | Small mobile buttons and labels can reduce the task effectiveness and efficiency. |

# Smartphone Features with associated Usability Limitations

| | |
|---|---|
| **Costly bandwidth and limited connectivity** | The problem of slow and poor internet connection can reduce the tasks efficiency. |
| **Wide heterogeneity** | Users of mobile systems continuously adapt to new forms. They need to learn different actions, letters, gestures and others. |
| **Mobility** | User while moving around may distract due to different context of use including bad light, noise and others. These constraints effects the application usability. |
| **kind of interaction** | Multitouch gesture is also not defined in a well-organised way. |
| **Interruption** | Notifications and requests may reduce the task efficiency and user's comfort level. |
| **Intimacy and availability** | User demands to available all the time. |
| **No hover state** | No hover state may arise the problem of the wrong click. |
| **Multi-device access** | User's familiarity with multi-device access demands mental effort to understand layout. |
| **Poor color contrast** | It decreases the visibility of the screen. |
| **Context-awareness** | CAMA do not properly sense the environment and behave differently. This may cause user's disturbance and misunderstanding about the displayed information. |
| **Wearable devices limited support** | All the given wearable, smart glass devices and other accessories are facing the usability issues. |

# Usability Heuristics for Smartphone

- Visibility of the system status
- *Definition*
  - The *smartphone* should keep the user informed about all processes and state changes through feedback and in a reasonable time. Feedback should facilitate user in under- standing the device's behavior.
- *Explanation*
  - All such changes should be visible and communicated to the user. There are some situations in which the user doesn't initiate the task, but system provides feedback in case of audio messages, low battery status and others. System provides feedback in different forms including sound, light color changes and others.

# Visibility of the system status

- *Example:* Battery is showing the status that it is left 72%.
- *Benefits:*
  - Users feel comfort and react accordingly upon precise feedback. Users well aware about any application or system state changes within time limit.
- *Problems:*
  - There are many different types of feedback in which the user's familiarity with all these does matter. Message sound generates against text messages, notifications, Whatsapp messages/audio, and others. So, different sounds recognition and familiarity among other types of feedback become difficult to understand and differentiate.



# Match between system and the real world

- *Definition*
  - The smartphone should express users' language instead of system oriented designs and technicalities. The smartphone should present the information in a natural and balanced way. Smartphone should follow the real world pacts.

- *Explaination*
- Displayed Information is of images, text, icons, titles, font, style, and color scheme follow conventions and should present according to the situation. It is observed that due to non- effective color scheme, looks difficult for users to select any specific option/ icon. So, system should design keeping in view environment variance. Use of color scheme and combination should conform to community belief, values, culture and environment as well as to enhance UX.

# Match between system and the real world

- *Example:*

Battery status and weather condition shows symbol of battery (bin) and clouds resembles respectively and according to the real world.

- *Benefits:*

Logical and natural arrangement of information supports every user to let it understand easily and perform the task efficiently.

- *Problems:*

Real world conventions vary due to different of beliefs,

culture, conventions and others. So, possibly, due to subjective nature, this heuristic identifies a flaw but, that wouldn’t be a flaw by some other community

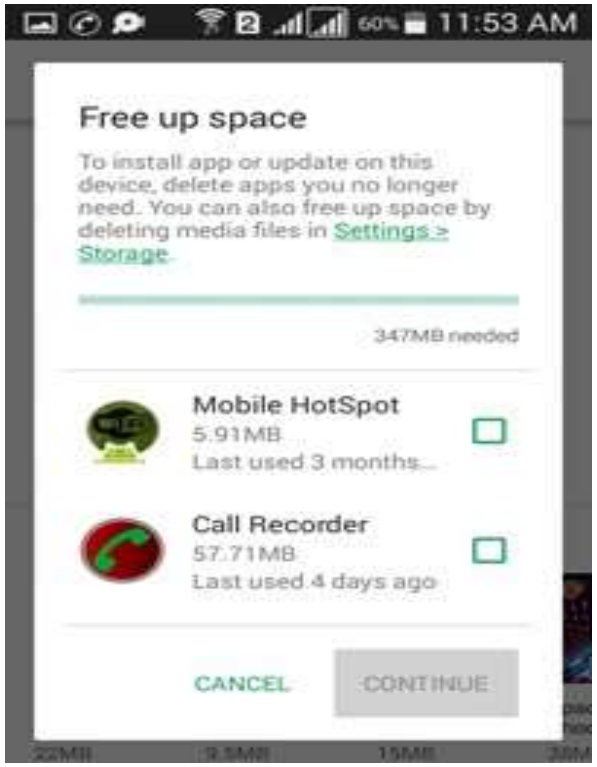


# Realistic error management

- *Definition*
- Users may not always be precise in performing the task on mobile devices due to many reasons, such as small screen size, kind of interaction etc. Application should provide mechanism to prevent from error as well as display error message with proper diagnosis and suggest suitable solution.
- *a: Explanation*
- System should provide proper security mechanism in case of sudden exit from the application domain, removing SIM/battery/sudden shutdown due to battery end up. In all such situations, system should provide a proper error message which highlights the consequences of the action and also provide a way to target situation in a good manner. When battery level goes down at a certain level, smartphone inform the user to take appropriate action such as enabling the power saving mode to save data and for long time usage.
- *b: Example*
- Smartphone displays error prevention mechanism, when the user wants to install or update application in case of low space. System also suggests appropriate solution to get free space in a good way as shown

# Realistic error management

- *c: Benefits*
- Error prevention leads to enhance user comfort and less time to complete the task. Clear and recognizable error messages support the user to handle the situation just like a technician and recover from the error
- *d: Problems*
- This heuristic instructs the user to understand the nature and type of error and how to use the system to come out of the danger situation. All this demands to know the workflow of the system. This may be less helpful for the naïve users.



# Help and Documentation

- The smartphone application should provide documentation and help focused on the user's current task.

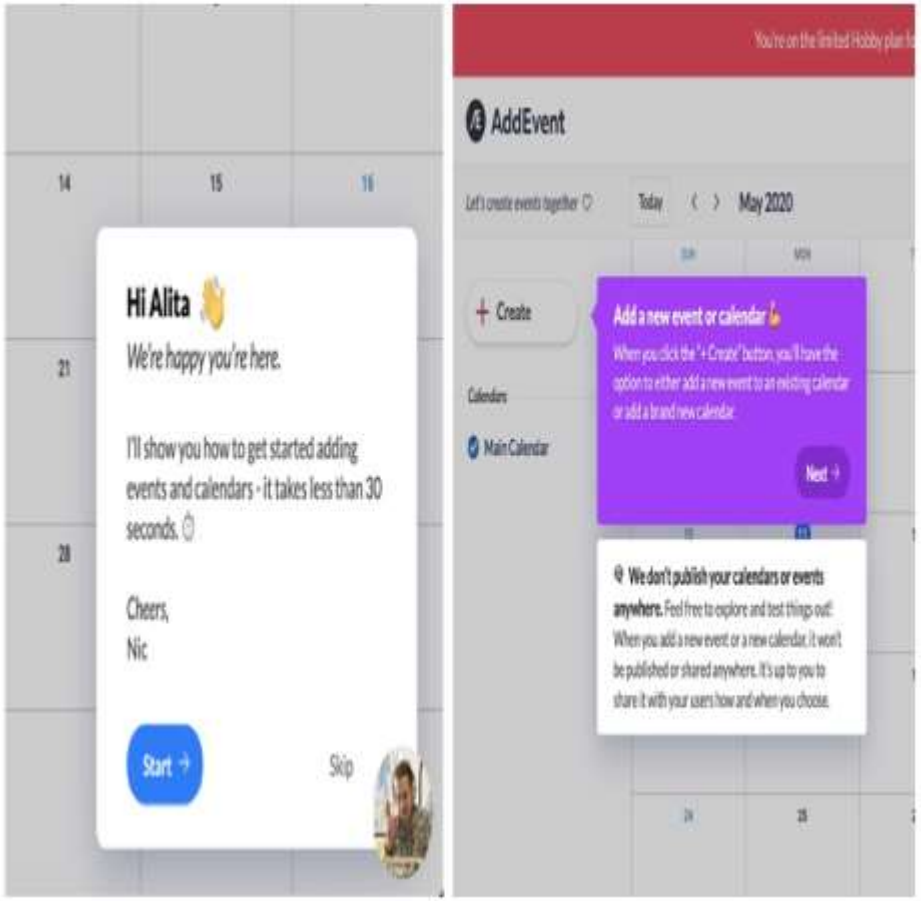
- *a: Explanation*
- Due to small screen size, designers put fewer information on screen. It may lead huge possibility to lose its meanings for the user. That's why, system should support the user to understand and execute the current task effectively and efficiently. One possible support is to provide proper help and documentation in an effective way. System should provide help and documentation with less cognitive work- load and context switching. Help and documentation may not necessarily organize in different menu setup or tab in mobile paradigm. System should support the user and provide help in the form of clues about next steps of task accomplishment.

- *b: Example*
- Smartphone provides proper help, when the user wants to display next image. Clear steps are mentioned to perform the current task. Another example is while using amazon mobile app, an option to add an item in the cart gives a tip to press and drag the item at specific place.

# Help and Documentation

- *c: Benefits*
- Proper help supports the user to accomplish the task effectively and efficiently. This heuristic also provides support to the user to get complete understanding of the system and can execute task with less error.

- *d: Problems*
- This heuristic supports the user about how to execute the current task for proper accomplishment. So, for naïve user, it is relatively difficult to understand and recognize.



# Refferences


- https://en.wikipedia.org/wiki/Usability
- https://nimitmangal.wordpress.com/2013/09/19/what-is-usability/
- https://learn.org/articles/What_is_Usability_Engineering.html
- https://ois2.ttu.ee › uusois/USABILITYandUSER INTERFACE DESIGN